

A2_python copy

November 2, 2025

```
[ ]: import os
import numpy as np
import pandas as pd
from scipy.interpolate import interp1d
from scipy.signal import savgol_filter
from scipy.io import loadmat
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
[ ]: # Functions I am adding
import modin.pandas as mpd
import ray

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.ar_model import AutoReg
from statsmodels.tsa.stattools import adfuller, acf
ray.init(ignore_reinit_error=True, num_cpus=os.cpu_count()-1)
```

2025-11-01 23:59:52,756 INFO worker.py:1718 -- Calling ray.init() again after it has already been called.

```
[ ]: RayContext(board_url='', python_version='3.11.11', ray_version='2.46.0',
ray_commit='c3dd2ca0c2a24ddf327a213d2e936bd4eaa4ca0a')
```

1 Read raw data

```
[3]: # Load label names
label_names = loadmat('label_names.mat', squeeze_me=True)
activity_names_indexed = label_names['activity_names_indexed']
```

```
[ ]: # Set constant variables
SAMPLE_RATE = 32
WINDOW_SIZE = 4 * 128
TIME_EXCLUDE = 2
CHUNK = 128
my_netid = "l01404"
```

```

# Loading and Processing Helper Functions
def load_label_info():
    """Load label names and create mapping from name+number."""
    label_data = loadmat("label_names.mat", squeeze_me=True)
    activity_names = label_data["activity_names_indexed"]
    mapping = {name: idx for idx, name in enumerate(activity_names, start=1)}
    return activity_names, mapping

def parse_filename(file_name, dir_name):
    """Extract user ID, file prefix, and activity name from filename."""
    substr = dir_name[dir_name.find("allData/") + 8:]
    prefix_cut = 11 + len(substr)
    if file_name.startswith("null"):
        activity_name = file_name[25:-prefix_cut]
    else:
        activity_name = file_name[36:-prefix_cut]
    file_common = file_name[:-(10)]
    return substr, file_common, activity_name

def load_accel_data(file_path):
    """Load and clean accelerometer data."""
    data = np.loadtxt(file_path, delimiter=",")
    ts = data[:, 0]
    dup_idx = np.where(ts[:-1] == ts[1:])[0]
    data = np.delete(data, dup_idx, axis=0)
    df = pd.DataFrame(data, columns=["ts", "accel_x", "accel_y", "accel_z"])
    df = df.sort_values("ts").interpolate(method="spline", order=2)
    return df

def load_baro_data(file_path, target_len):
    """Load, deduplicate, interpolate, and smooth barometer data."""
    bar_data = np.loadtxt(file_path, delimiter=",")
    ts_bar = bar_data[:, 0]
    dup_idx = np.where(ts_bar[:-1] == ts_bar[1:])[0]
    bar_data = np.delete(bar_data, dup_idx, axis=0)
    interp_idx = np.linspace(0, len(bar_data) - 1, target_len)
    interp_func = interp1d(np.arange(len(bar_data)), bar_data[:, 1],
                           kind="linear", fill_value="extrapolate")
    y_bar = interp_func(interp_idx)
    y_bar_smoothed = savgol_filter(y_bar, WINDOW_SIZE, 1, mode="nearest")
    return y_bar_smoothed

def trim_data(df, y_bar):
    """Trim first/last few seconds from both accel and barometer."""
    start = int(TIME_EXCLUDE * SAMPLE_RATE)
    end = -start if start > 0 else None

```

```

df_trim = df.iloc[start:end]
y_bar_trim = y_bar[start:end]
return df_trim, y_bar_trim

def prepare_fixed_blocks(df_accel, y_bar, activity, user, label):
    """Ensure dataset is multiple of CHUNK=128 samples."""
    usable_len = CHUNK * (len(y_bar) // CHUNK)
    if usable_len <= 0:
        return pd.DataFrame()
    df_accel = df_accel.iloc[:usable_len].copy()
    df_accel["y_bar"] = y_bar[:usable_len]
    df_accel["activity"] = activity
    df_accel["user"] = user
    df_accel["label"] = label
    return df_accel

# Parallel Worker
@ray.remote
def process_file(file_name, dir_name, activity_names, mapping):
    """Process one accel file (and its matching pressure file)."""
    substr, file_common, activity_name = parse_filename(file_name, dir_name)
    if activity_name not in activity_names:
        return pd.DataFrame()
    accel_path = os.path.join(dir_name, file_name)
    baro_path = os.path.join(dir_name, file_common + "-pressure.txt")
    df_accel = load_accel_data(accel_path)
    y_bar = load_baro_data(baro_path, len(df_accel))
    df_accel, y_bar = trim_data(df_accel, y_bar)
    df_out = prepare_fixed_blocks(df_accel, y_bar,
                                  activity_name, substr, mapping[activity_name])
    return df_out

# Main per-user computation
def compute_raw_data_parallel(dir_name):
    """Parallelized raw data extraction for a single user directory."""
    activity_names, mapping = load_label_info()
    files = [f for f in os.listdir(dir_name) if f.endswith("-accel.txt")]
    tasks = [process_file.remote(f, dir_name, activity_names, mapping) for f in files]
    results = ray.get(tasks)
    if not results:
        return pd.DataFrame()
    return pd.concat(results, ignore_index=True)

# Iterate over all user directories
def build_full_dataset(base_dir):

```

```

    """Iterate over all user directories under 'allData/' and combine results.
    """
    col_names = ['ts', 'accel_x', 'accel_y', 'accel_z', 'y_bar', 'label',
    'user', 'activity']
    df_raw = pd.DataFrame(columns=col_names)

    for data_dir_name in os.listdir(base_dir):
        user_path = os.path.join(base_dir, data_dir_name)
        if os.path.isdir(user_path) and not data_dir_name.startswith("."):
            print(f"Processing directory: {data_dir_name}")
            df_temp = compute_raw_data_parallel(user_path)

            # Only sort/append if non-empty and contains expected columns
            if not df_temp.empty and all(col in df_temp.columns for col in
            ['label', 'ts']):
                df_temp = df_temp.sort_values(by=['label', 'ts'],
            ignore_index=True)
                df_raw = pd.concat([df_raw, df_temp], ignore_index=True)
            else:
                print(f"Skipping {data_dir_name}: no valid data found")

    return df_raw

# Main Pipeline
def compute_raw_data_parallel(dir_name):
    """Parallelized and modularized raw data extraction pipeline."""
    activity_names, mapping = load_label_info()
    files = [f for f in os.listdir(dir_name) if f.endswith("-accel.txt")]

    tasks = [
        process_file.remote(f, dir_name, activity_names, mapping)
        for f in files
    ]
    results = ray.get(tasks)
    return pd.concat(results, ignore_index=True)

```

```

[9]: # Run the full pipeline
data_dir = os.path.join(os.getcwd(), "allData")
df_raw = build_full_dataset(data_dir)

# Keep only the desired columns (and ensure order)
col_names_raw = ['ts', 'accel_x', 'accel_y', 'accel_z', 'y_bar', 'label',
    'user', 'activity']
df_raw = df_raw[[col for col in col_names_raw if col in df_raw.columns]]

print(f"Final dataset shape: {df_raw.shape}")
df_raw

```

Processing directory: vbagdi
 Processing directory: ahs264
 Processing directory: cyt25
 Skipping cyt25: no valid data found
 Processing directory: xl549
 Processing directory: jl2855
 Processing directory: xz278
 Processing directory: johnsweeney
 Skipping johnsweeney: no valid data found
 Processing directory: Grack
 Processing directory: mmill
 Processing directory: mrossman
 Processing directory: cstewie
 Processing directory: sutreja
 Processing directory: sln1
 Processing directory: yinuoinno
 Processing directory: jao57
 Skipping jao57: no valid data found
 Processing directory: jude
 Processing directory: aaleksanyan
 Processing directory: yaw006
 Processing directory: aks249
 Processing directory: mmb289
 Processing directory: av452
 Processing directory: sp2333
 Processing directory: nbw32
 Processing directory: nk428
 Processing directory: mlc299
 Processing directory: cgp46
 Processing directory: wh298
 Processing directory: s3chowdhury
 Processing directory: jps394
 Processing directory: shrek1611
 Processing directory: bl4521
 Processing directory: amahdin
 Processing directory: lol404
 Processing directory: mythili
 Processing directory: ktc36
 Processing directory: xichen
 Final dataset shape: (7091072, 8)

```

[9]:          ts    accel_x  accel_y    accel_z      y_bar label \
0      1.761522e+12 -0.443601 -0.537366    9.946635 1001.999464    1
1      1.761522e+12 -0.381671 -0.494507   10.008340 1001.999263    1
2      1.761522e+12 -0.415024 -0.532604    9.908663 1001.999056    1
3      1.761522e+12 -0.395971 -0.513556    9.908663 1001.998843    1
4      1.761522e+12 -0.386446 -0.513557    9.918155 1001.998624    1
  
```

```

...
7091067  1.760986e+12  2.644069 -0.965504  7.988439  1004.050420  7
7091068  1.760986e+12  2.654238 -0.922433  7.983055  1004.051759  7
7091069  1.760986e+12  2.673979 -0.869791  8.021339  1004.053094  7
7091070  1.760986e+12  2.674577 -0.816551  8.044670  1004.054423  7
7091071  1.760986e+12  2.660819 -0.744766  8.059027  1004.055747  7

```

```

      user      activity
0    vbagdi  Stationary
1    vbagdi  Stationary
2    vbagdi  Stationary
3    vbagdi  Stationary
4    vbagdi  Stationary

```

```

...
7091067  xichen  Elevator-down
7091068  xichen  Elevator-down
7091069  xichen  Elevator-down
7091070  xichen  Elevator-down
7091071  xichen  Elevator-down

```

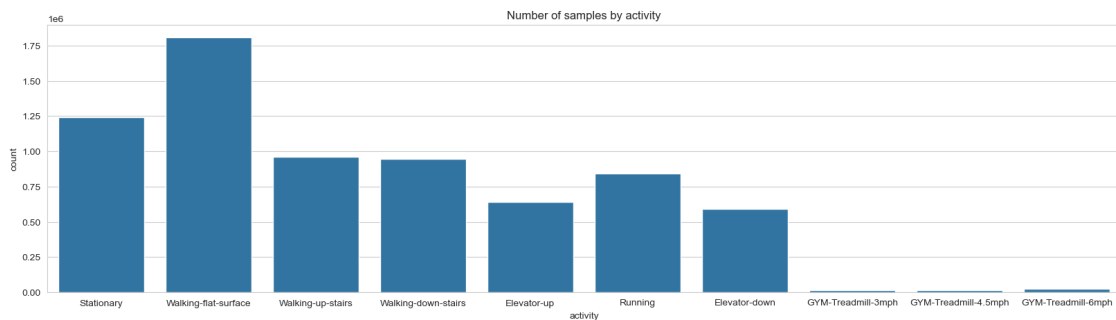
[7091072 rows x 8 columns]

2 Visualize raw data

```

[10]: # Distribution of activities
sns.set_style("whitegrid")
plt.figure(figsize = (20, 5))
sns.countplot(x = "activity", data = df_raw)
plt.title("Number of samples by activity")
plt.show()

```

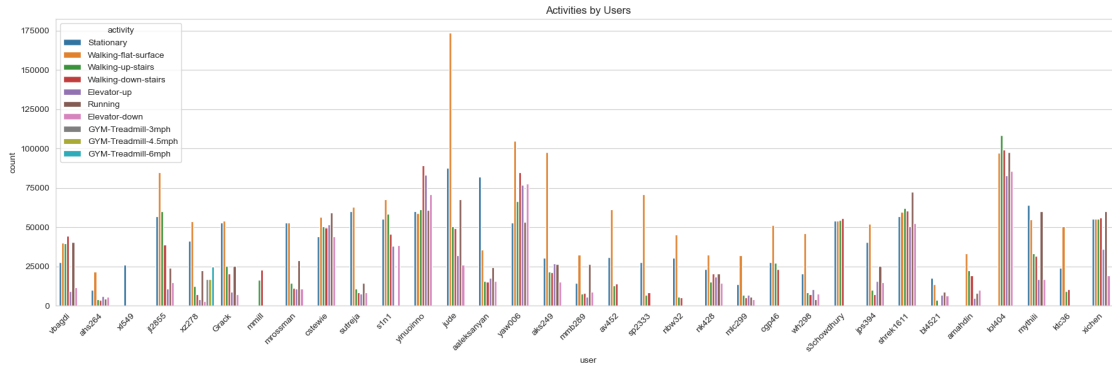


```

[ ]: # Distribution of activities among different users
plt.figure(figsize=(18, 6))
sns.countplot(x="user", hue="activity", data=df_raw)

```

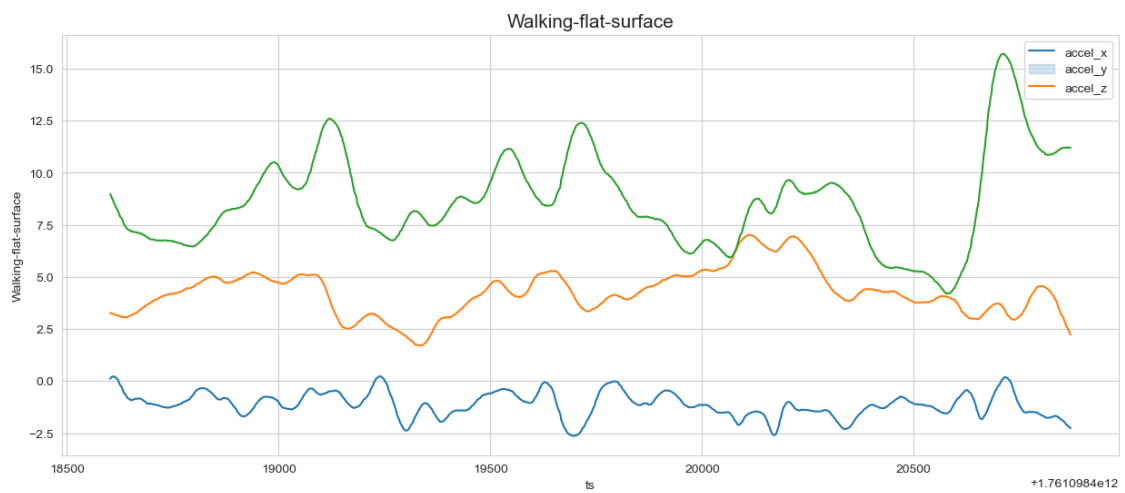
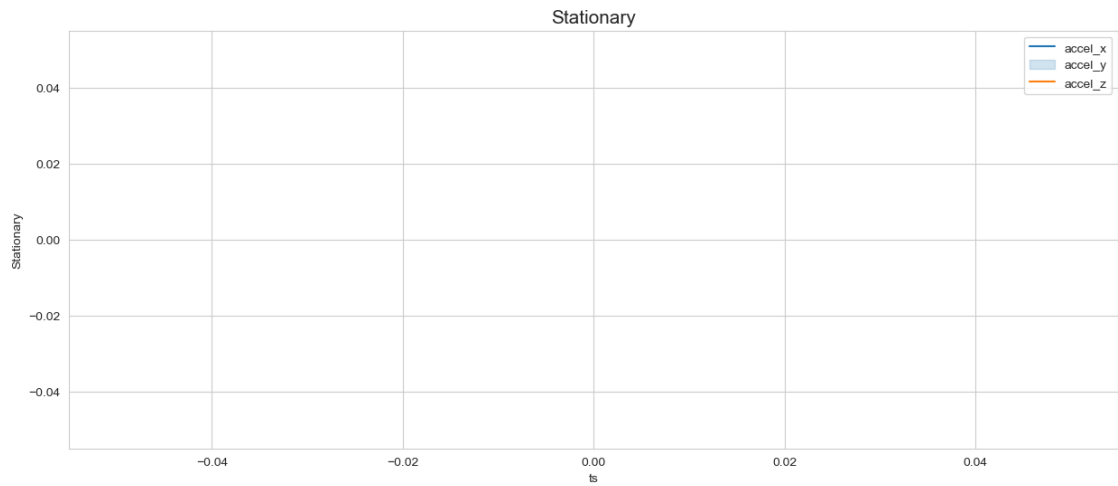
```
plt.title("Activities by Users")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



```
[13]: # splitting train data and test data
df_train = df_raw[df_raw['user'] != my_netid]
df_test = df_raw[df_raw['user'] == my_netid]
```

```
[14]: # Visual representation of raw data
# Note how the magnitudes and signal varies for each activity

for i in ["Stationary", "Running", "Walking-flat-surface"]:
    data_x = df_raw[(df_raw["user"] == my_netid) & (df_raw["activity"] == i)][:
↪1000]
    plt.figure(figsize = (15, 6))
    sns.lineplot(y = "accel_x", x = "ts", data = data_x)
    sns.lineplot(y = "accel_y", x = "ts", data = data_x)
    sns.lineplot(y = "accel_z", x = "ts", data = data_x)
    plt.legend(["accel_x", "accel_y", "accel_z"])
    plt.ylabel(i)
    plt.title(i, fontsize = 15)
    plt.show()
```




```
[ ]: # Build dataframes
df_train_windows = windows_to_dataframe(train_result)
df_test_windows = windows_to_dataframe(test_result)

# Combine into one master DataFrame
df_windows_all = pd.concat([df_train_windows, df_test_windows],
    ↪ ignore_index=True)
df_windows_all
```

```
[ ]:      user  label      ts_start      ts_end  \
0      vbagdi      1  1.761522e+12  1.761522e+12
1      vbagdi      1  1.761522e+12  1.761522e+12
2      vbagdi      1  1.761522e+12  1.761522e+12
3      vbagdi      1  1.761522e+12  1.761522e+12
4      vbagdi      1  1.761522e+12  1.761522e+12
...      ...      ...
141814  lol1404      7  1.761099e+12  1.761099e+12
141815  lol1404      7  1.761099e+12  1.761099e+12
141816  lol1404      7  1.761099e+12  1.761099e+12
141817  lol1404      7  1.761099e+12  1.761099e+12
141818  lol1404      7  1.761099e+12  1.761099e+12

                                accel_x  \
0      [-0.4436013, -0.3816712, -0.41502357, -0.39597...
1      [-0.45330125, -0.4151919, -0.4294876, -0.40567...
2      [-0.4296559, -0.37248993, -0.439191, -0.453486...
3      [-0.42029935, -0.45841578, -0.49176803, -0.434...
4      [-0.45381987, -0.439531, -0.48717573, -0.47288...
...
141814  [-3.0939198, -3.0987053, -3.0897322, -3.099303...
141815  [-3.3702908, -3.365505, -3.3559337, -3.3607194...
141816  [-4.3112683, -4.3208394, -4.330411, -4.325027,...
141817  [-3.964309, -3.9649072, -3.9511483, -3.955934,...
141818  [-4.2592244, -4.2592244, -4.2681975, -4.28734,...

                                accel_y  \
0      [-0.5373656, -0.4945072, -0.5326041, -0.513556...
1      [-0.5611921, -0.54214406, -0.53262025, -0.5421...
2      [-0.49930152, -0.5183501, -0.49454015, -0.5088...
3      [-0.5517004, -0.58027315, -0.5802735, -0.56122...
4      [-0.53266823, -0.4802858, -0.4659999, -0.45647...
...
141814  [5.580661, 5.580661, 5.5519466, 5.523233, 5.53...
141815  [5.54417, 5.54417, 5.548956, 5.5537415, 5.5489...
```

```

141816 [5.7924256, 5.7924256, 5.8073807, 5.8217373, 5...
141817 [5.8097734, 5.7762737, 5.761917, 5.757131, 5.7...
141818 [5.5262237, 5.5220366, 5.5316076, 5.541777, 5...

                                accel_z  \
0      [9.946635, 10.00834, 9.908663, 9.908663, 9.918...
1      [9.932384, 9.92289, 9.932384, 9.937129, 10.008...
2      [9.96085, 9.918132, 9.94661, 9.899144, 9.87541...
3      [9.884893, 9.880146, 9.946598, 9.913371, 9.956...
4      [9.870642, 9.894374, 9.941838, 9.956078, 9.965...
...
141814 [7.938189, 7.966903, 7.9908314, 8.014161, 8.00...
141815 [7.2502527, 7.1694946, 7.0839515, 7.0414786, 7...
141816 [6.7184477, 6.689734, 6.6514487, 6.613762, 6.5...
141817 [6.7848487, 6.789634, 6.8039913, 6.8129644, 6...
141818 [7.079764, 7.0319076, 6.9894347, 6.9373913, 6...

                                y_bar    set
0      [1001.9994642575144, 1001.9992632181882, 1001...  train
1      [1001.9888634688094, 1001.988659832066, 1001.9...  train
2      [1001.9767912092564, 1001.9765119436063, 1001...  train
3      [1001.9650532552037, 1001.9648426794869, 1001...  train
4      [1001.9547192782265, 1001.9545467688555, 1001...  train
...
141814 [1002.6069473070337, 1002.6061259618702, 1002...  test
141815 [1002.5729640392408, 1002.5723195531115, 1002...  test
141816 [1002.5517744701326, 1002.5518262533452, 1002...  test
141817 [1002.5527393588654, 1002.552931790506, 1002.5...  test
141818 [1002.5632562400754, 1002.5632281598541, 1002...  test

[141819 rows x 9 columns]

```

3 Check ARMA assumptions

3.1 Stationarity

3.1.1 Visualize user process

```

[ ]: def plot_user_activity(df, user_id=None, activity_name="Walking-flat-surface",
    ↪random=True):
    """
    Plot accel_x, accel_y, accel_z, and barometric pressure for a given user_
    ↪and activity.
    If random=True, picks a random user who performed that activity.

    Parameters:
    df (pd.DataFrame): The full dataset (df_raw)

```

```

        user_id (str): Specific user ID (ignored if random=True)
        activity_name (str): Activity name (e.g. 'Walking-flat-surface')
        random (bool): Whether to pick a random user instead of specifying one
    """
    # Pick random user if requested
    if random:
        users = df[df["activity"] == activity_name]["user"].unique()
        user_id = np.random.choice(users)
        print(f"Randomly selected user: {user_id}")

    # Filter
    df_user = df[(df["user"] == user_id) & (df["activity"] == activity_name)]

    # Extract signals
    accel_x = df_user["accel_x"].dropna().reset_index(drop=True)
    accel_y = df_user["accel_y"].dropna().reset_index(drop=True)
    accel_z = df_user["accel_z"].dropna().reset_index(drop=True)
    baro = df_user["y_bar"].dropna().reset_index(drop=True)

    # Plot
    fig, axes = plt.subplots(4, 1, figsize=(14, 8), sharex=True)

    axes[0].plot(accel_x, color='tab:blue')
    axes[0].set_title(f"Accel_x for {user_id} ({activity_name})")
    axes[0].set_ylabel("Accel X")

    axes[1].plot(accel_y, color='tab:orange')
    axes[1].set_title("Accel_y")
    axes[1].set_ylabel("Accel Y")

    axes[2].plot(accel_z, color='tab:green')
    axes[2].set_title("Accel_z")
    axes[2].set_ylabel("Accel Z")

    axes[3].plot(baro, color='tab:red')
    axes[3].set_title("Barometric Pressure")
    axes[3].set_xlabel("Time index")
    axes[3].set_ylabel("Pressure")

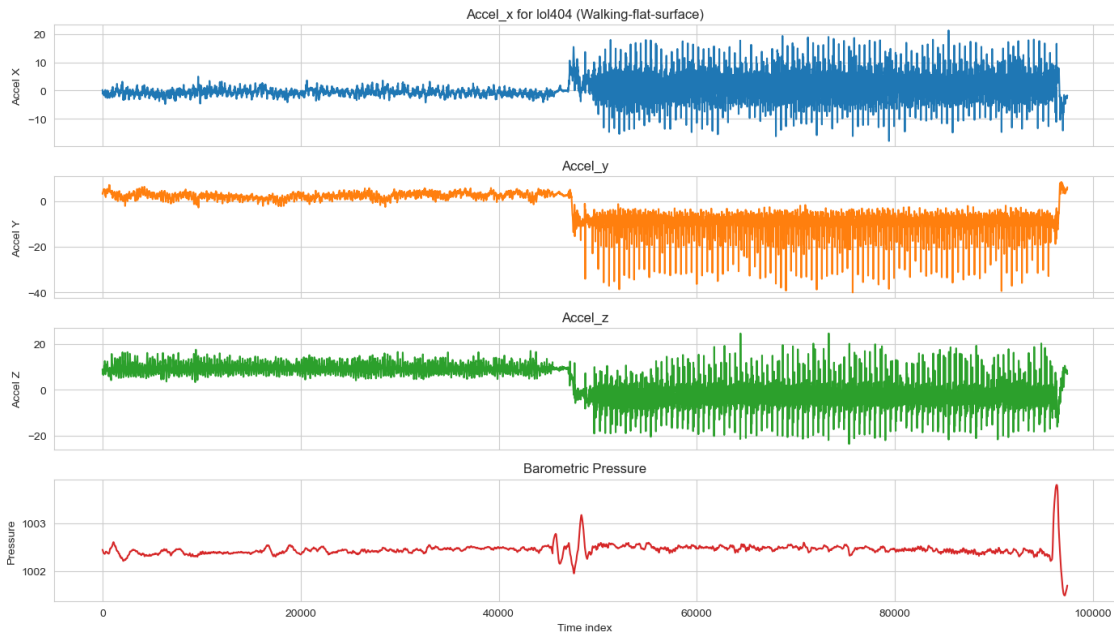
    plt.tight_layout()
    plt.show()

```

```

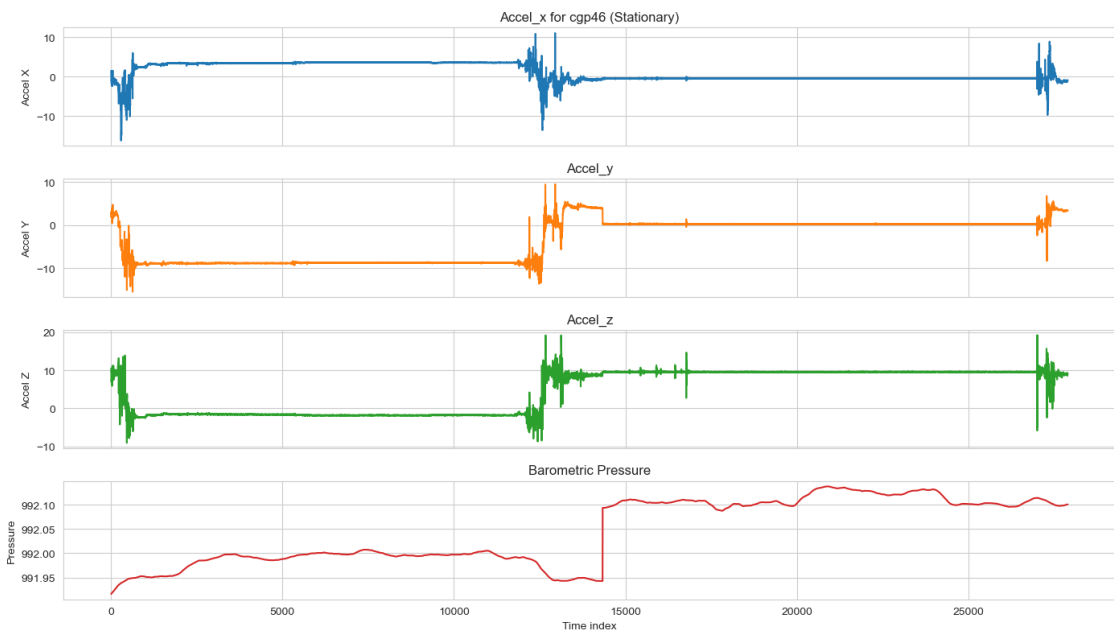
[100]: activity_walk = "Walking-flat-surface"
       plot_user_activity(df_raw, my_netid, activity_walk, random = False)

```



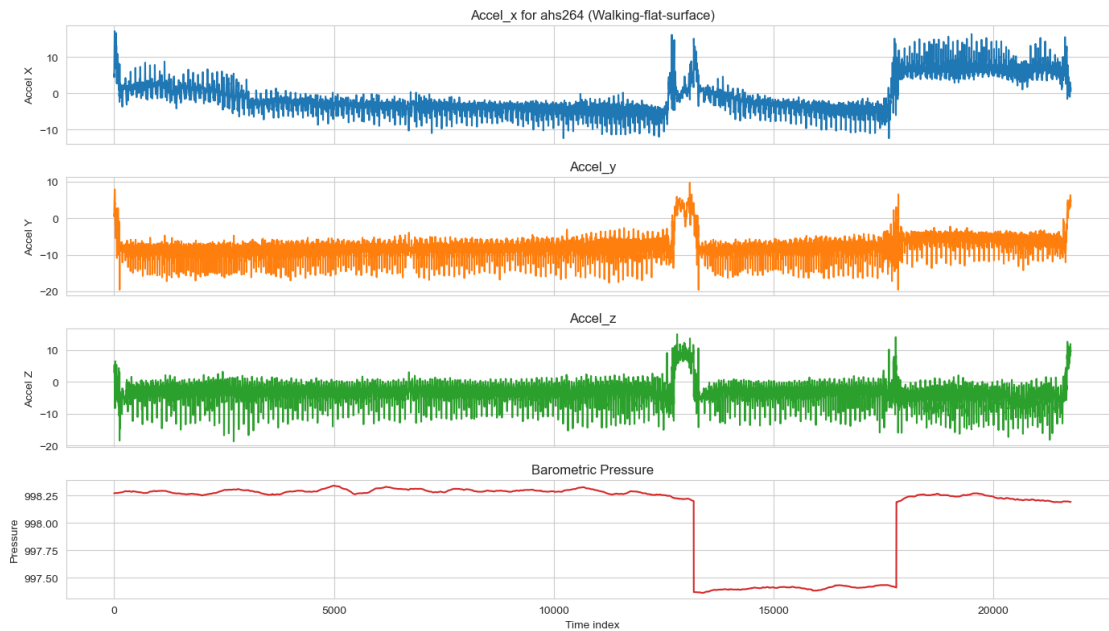
```
[ ]: plot_user_activity(df_raw, activity_walk, random=True)
```

Randomly selected user: cgp46



```
[ ]: plot_user_activity(df_raw, activity_walk, random=True)
```

Randomly selected user: ahs264



4 Processing

5 Split by sessions and trim

```
[ ]: # Adaptive threshold calculation
def get_adaptive_gap_thresholds(df):
    """
    Build per-(user,activity) gap thresholds based on  $\Delta t$ .
    Caps to [100, 10000] ms (0.1-10 seconds).
    """
    summary = []
    for uid in df["user"].unique():
        for act in df["activity"].unique():
            d = df[(df["user"] == uid) & (df["activity"] == act)]
            if len(d) < 10:
                continue
            ts = d["ts"].astype(float).values
            dt = np.diff(ts)
            if len(dt) == 0:
                continue

            med = np.median(dt)
            max_dt = np.max(dt)
            thr = np.clip(50 * med, 100, 10000) # 50x median capped 0.1-10 s
```

```

summary.append({
    "user": uid,
    "activity": act,
    "median_dt": med,
    "max_dt": max_dt,
    "gap_threshold_ms": thr
})

thresholds = pd.DataFrame(summary)
print(f"Adaptive thresholds for {len(thresholds)} user-activity pairs")
return thresholds

# 2. Find sessions based on timestamp gaps
def find_sessions(df_user, gap_threshold_ms=5000):
    """Split into sessions based on large timestamp gaps."""
    df_user = df_user.sort_values("ts").reset_index(drop=True)
    ts = df_user["ts"].astype(float).values
    dt = np.diff(ts)

    cut_idx = np.where(dt > gap_threshold_ms)[0]
    starts = np.r_[0, cut_idx + 1]
    ends = np.r_[cut_idx + 1, len(df_user)]
    return list(zip(starts, ends))

# 3. Trim sessions (10% each side)
def trim_session(session_df, trim_frac=0.10):
    """Trim a fraction of samples off each side of one session."""
    n = len(session_df)
    if n < 200:
        return pd.DataFrame()

    start = int(n * trim_frac)
    end = int(n * (1 - trim_frac))
    trimmed = session_df.iloc[start:end].reset_index(drop=True)

    # Drop empty or all-NaN sessions
    if trimmed.empty or trimmed.isna().all().any():
        return pd.DataFrame()

    return trimmed

# 4. Ray worker: process one user/activity
@ray.remote
def process_user_activity(df, user_id, activity, threshold_map, trim_frac=0.10):
    d = df[(df["user"] == user_id) & (df["activity"] == activity)].copy()
    if d.empty:
        return pd.DataFrame()

```

```

gap_thr = threshold_map.get((user_id, activity), 3000)
session_ranges = find_sessions(d, gap_threshold_ms=gap_thr)
if not session_ranges:
    return pd.DataFrame()

all_sessions = []
for i, (s, e) in enumerate(session_ranges, 1):
    seg = trim_session(d.iloc[s:e], trim_frac)
    if not seg.empty and len(seg) > 200:
        seg["session_id"] = f"{user_id}_{activity}_session{i}"
        all_sessions.append(seg)

return pd.concat(all_sessions, ignore_index=True) if all_sessions else pd.
↳ DataFrame()

# 5. Combine across all users & activities
def split_and_trim_all_sessions(df, trim_frac=0.10):
    if not ray.is_initialized():
        ray.init(ignore_reinit_error=True)

    thresholds_df = get_adaptive_gap_thresholds(df)
    threshold_map = {
        (r.user, r.activity): r.gap_threshold_ms
        for _, r in thresholds_df.iterrows()
    }

    all_users = df["user"].unique()
    all_activities = df["activity"].unique()

    tasks = [
        process_user_activity.remote(df, uid, act, threshold_map, trim_frac)
        for uid in all_users
        for act in all_activities
    ]

    results = ray.get(tasks)
    valid = [r for r in results if not r.empty and "session_id" in r.columns]

    if not valid:
        print("No valid sessions found.")
        return pd.DataFrame()

    df_combined = pd.concat(valid, ignore_index=True)
    df_combined = df_combined[df_combined["session_id"].notna()]
    df_combined = df_combined.groupby("session_id").filter(lambda x: len(x) >
↳ 200)

```

```

print(f"Combined DataFrame: {len(df_combined)} samples, "
      f"{df_combined['session_id'].nunique()} sessions, "
      f"{df_combined['user'].nunique()} users, "
      f"{df_combined['activity'].nunique()} activities")

return df_combined

```

```

[722]: df_trimmedSessions = split_and_trim_all_sessions(df_raw)
df_trimmedSessions

```

Adaptive thresholds for 201 user-activity pairs

Combined DataFrame: 5671614 samples, 499 sessions, 33 users, 10 activities

```

[722]:
      ts      accel_x  accel_y  accel_z      y_bar label \
0      1.761522e+12 -0.410129 -0.595393  9.893749  1001.991366    1
1      1.761522e+12 -0.424429 -0.619204  9.960200  1001.991455    1
2      1.761522e+12 -0.448254 -0.595394  9.912735  1001.991545    1
3      1.761522e+12 -0.438729 -0.547774  9.884255  1001.991635    1
4      1.761522e+12 -0.433973 -0.585871  9.907987  1001.991725    1
...
5671609  1.760986e+12  2.286940 -9.455837 -0.574277  1004.803224    7
5671610  1.760986e+12  2.355136 -9.322437 -0.665803  1004.803822    7
5671611  1.760986e+12  2.401796 -9.395418 -0.736989  1004.804398    7
5671612  1.760986e+12  2.440081 -9.372088 -0.775274  1004.804954    7
5671613  1.760986e+12  2.446661 -9.266206 -0.751945  1004.805489    7

```

```

      user      activity      session_id
0      vbagdi      Stationary  vbagdi_Stationary_session1
1      vbagdi      Stationary  vbagdi_Stationary_session1
2      vbagdi      Stationary  vbagdi_Stationary_session1
3      vbagdi      Stationary  vbagdi_Stationary_session1
4      vbagdi      Stationary  vbagdi_Stationary_session1
...
5671609  xichen  Elevator-down  xichen_Elevator-down_session2
5671610  xichen  Elevator-down  xichen_Elevator-down_session2
5671611  xichen  Elevator-down  xichen_Elevator-down_session2
5671612  xichen  Elevator-down  xichen_Elevator-down_session2
5671613  xichen  Elevator-down  xichen_Elevator-down_session2

```

[5671614 rows x 9 columns]

```

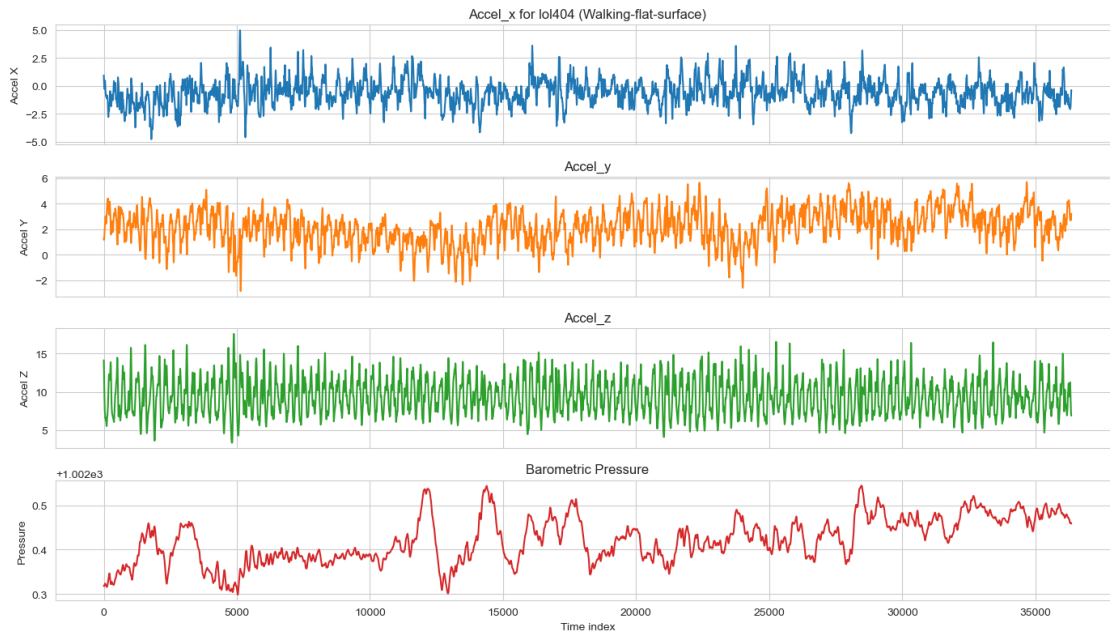
[723]: df_trimmedSessions.loc[
      (df_trimmedSessions["user"] == "lol404") &
      (df_trimmedSessions["activity"] ==
        ↪ "Walking-flat-surface"),
      "session_id"
].unique()

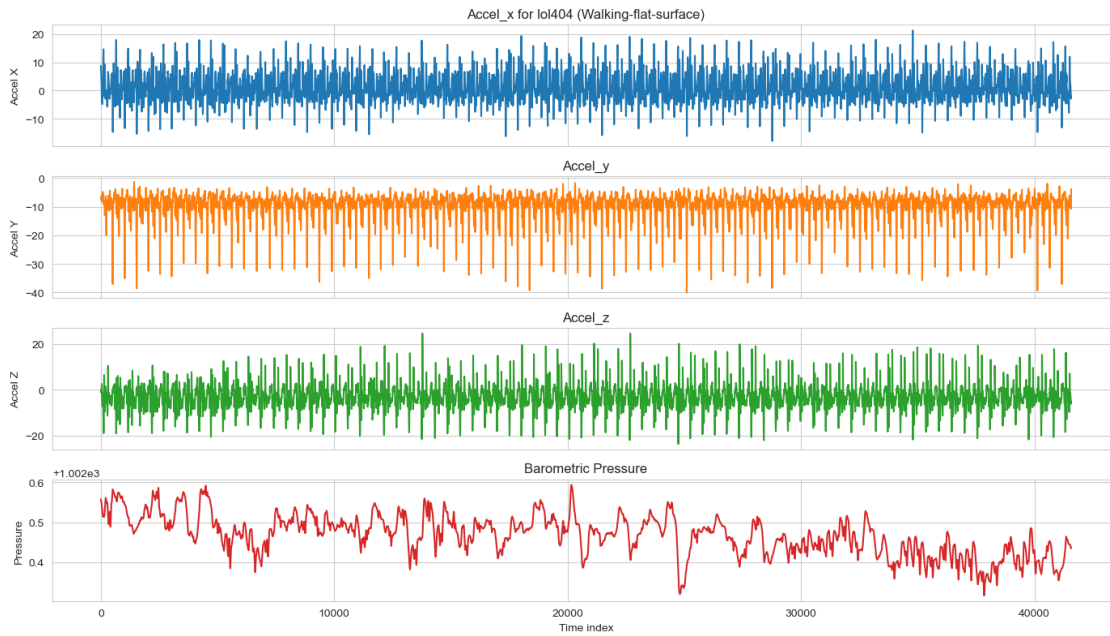
```



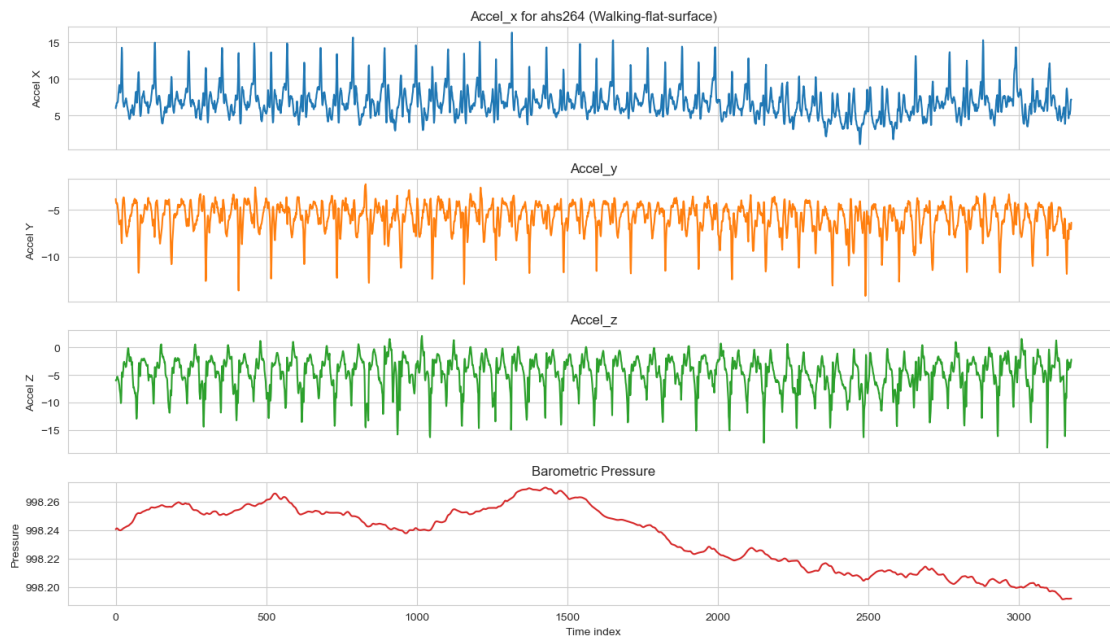
```
[723]: array(['lol404_Walking-flat-surface_session1',  
            'lol404_Walking-flat-surface_session2'], dtype=object)
```

```
[724]: sessions = [s for s in  
    ↳ df_trimmedSessions[df_trimmedSessions["user"]=="lol404"]["session_id"].  
    ↳ unique()[:2]]  
for s in sessions:  
    ↳ plot_user_activity(df_trimmedSessions[df_trimmedSessions["session_id"]==s],  
    ↳ "lol404", random=False)
```





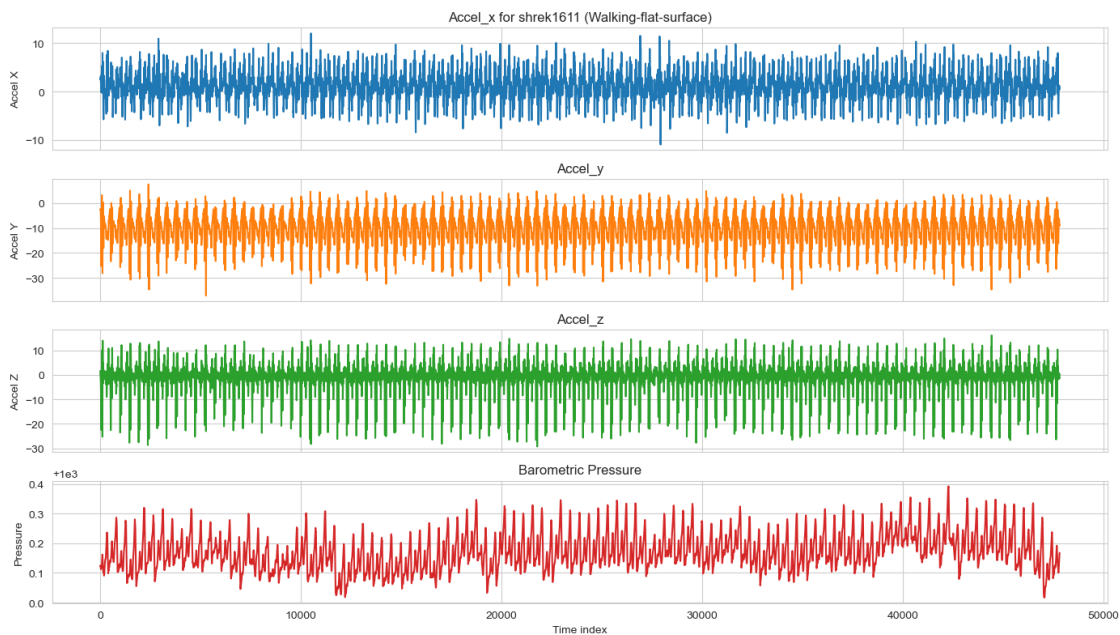
```
[735]: session_rand = np.random.choice(df_trimmedSessions["session_id"].unique())
df_rand = df_trimmedSessions[df_trimmedSessions["session_id"] == session_rand]
plot_user_activity(df_rand, user_id=df_rand["user"].iloc[0], random=False)
```



```
[736]: df_trimmedSessions["session_id"].nunique()
```

[736]: 499

```
[1187]: session_rand = np.random.choice(df_trimmedSessionsCleaned["session_id"].  
      ↪unique())  
df_rand = df_trimmedSessionsCleaned[df_trimmedSessionsCleaned["session_id"] ==  
      ↪session_rand]  
plot_user_activity(df_rand, user_id=df_rand["user"].iloc[0], random=False)
```



```
[1189]: session_rand = np.random.choice(df_trimmedSessionsSnip["session_id"].unique())  
df_rand = df_trimmedSessionsSnip[df_trimmedSessionsSnip["session_id"] ==  
      ↪session_rand]  
plot_user_activity(df_rand, user_id=df_rand["user"].iloc[0], random=False)
```

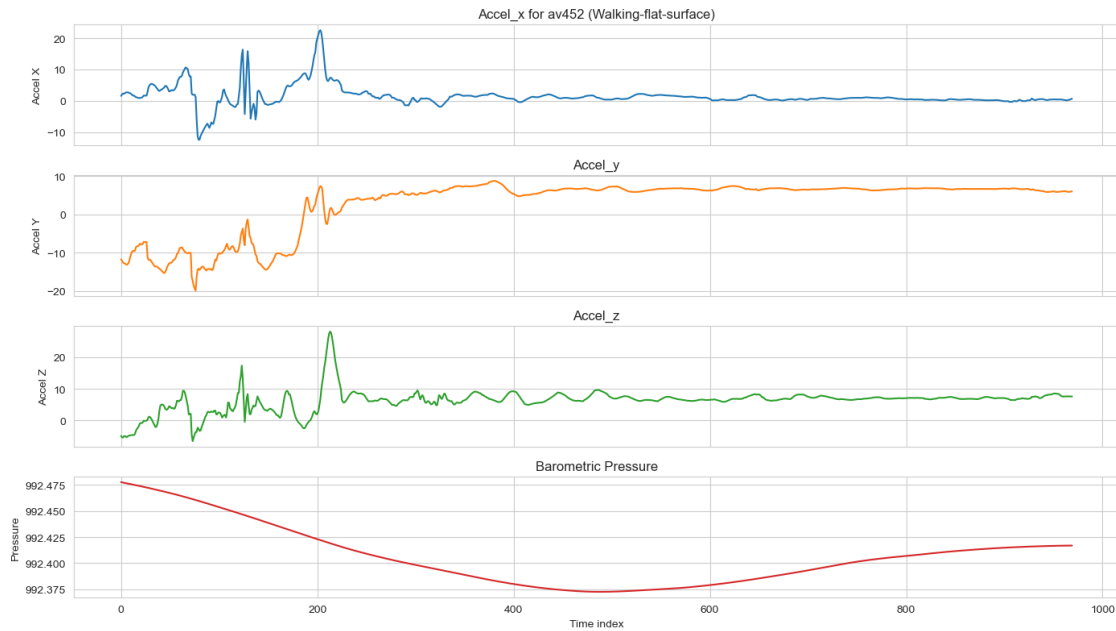


```
[ ]: # Choose a random activity
activity_rand = np.random.choice(df_trimmedSessions["activity"].unique())

# Choose a random session from that activity
sessions_in_activity = df_trimmedSessions.loc[
    df_trimmedSessions["activity"] == activity_rand, "session_id"
].unique()
session_rand = np.random.choice(sessions_in_activity)

# Extract and plot
df_rand = df_trimmedSessions[df_trimmedSessions["session_id"] == session_rand]
plot_user_activity(
    df_rand,
    user_id=df_rand["user"].iloc[0],
    activity_name=activity_rand,
    random=False
)

print(f"Randomly selected: {activity_rand} | {df_rand['session_id'].iloc[0]}")
```



Randomly selected: Walking-flat-surface | av452_Walking-flat-surface_session3

6 Windowing

```
[ ]: # Create overlapping windows per session
def create_overlapping_windows(df, window_size=100, step_size=50):
    """
    Create overlapping windows (with timestamps) for accel_x, accel_y, accel_z,
    y_bar.
    Works on trimmed sessions - keeps user, activity, and session_id context.
    """
    x_list, y_list, z_list, b_list = [], [], [], []
    labels, participants, timestamps, activities, sessions = [], [], [], [], []

    for i in range(0, df.shape[0] - window_size, step_size):
        window = df.iloc[i:i + window_size]

        # Core sensor arrays
        x_list.append(window['accel_x'].values)
        y_list.append(window['accel_y'].values)
        z_list.append(window['accel_z'].values)
        b_list.append(window['y_bar'].values)

        # Metadata
        timestamps.append((window['ts'].iloc[0], window['ts'].iloc[-1]))
```

```

        labels.append(window['label'].mode(dropna=True).iloc[0] if not
↪window['label'].mode(dropna=True).empty else None)
        participants.append(window['user'].mode(dropna=True).iloc[0] if not
↪window['user'].mode(dropna=True).empty else None)
        activities.append(window['activity'].mode(dropna=True).iloc[0] if not
↪window['activity'].mode(dropna=True).empty else None)
        sessions.append(window['session_id'].mode(dropna=True).iloc[0] if
↪'session_id' in window.columns and not window['session_id'].
↪mode(dropna=True).empty else None)

    return {
        "x": x_list, "y": y_list, "z": z_list, "b": b_list,
        "labels": labels, "participants": participants,
        "timestamps": timestamps, "activities": activities, "sessions": sessions
    }

#Ray remote worker
@ray.remote
def make_windows_remote(df, window_size=100, step_size=50):
    return create_overlapping_windows(df, window_size, step_size)

# Parallel window generation across all sessions
def generate_windows_all_sessions(df, window_size=100, step_size=50):
    """
    Generate overlapping windows for all sessions in parallel using Ray.
    Returns combined DataFrame of all windows.
    """
    if not ray.is_initialized():
        ray.init(ignore_reinit_error=True)

    session_ids = df["session_id"].unique()
    tasks = [make_windows_remote.remote(df[df["session_id"] == sid],
↪window_size, step_size)
              for sid in session_ids]

    results = ray.get(tasks)
    all_results = [r for r in results if len(r["x"]) > 0]

    # Combine into single DataFrame
    df_all = pd.concat([windows_to_dataframe(r) for r in all_results],
↪ignore_index=True)
    print(f"Generated {len(df_all)} windows from {len(session_ids)} sessions.")
    return df_all

```

```

# Convert to structured DataFrame
def windows_to_dataframe(result_dict):
    """
    Convert windowed sensor data into a structured DataFrame for modeling or
    ↪stationarity tests.
    """
    df = pd.DataFrame({
        "user": result_dict["participants"],
        "activity": result_dict["activities"],
        "session_id": result_dict["sessions"],
        "label": result_dict["labels"],
        "ts_start": [t[0] for t in result_dict["timestamps"]],
        "ts_end": [t[1] for t in result_dict["timestamps"]],
        "accel_x": result_dict["x"],
        "accel_y": result_dict["y"],
        "accel_z": result_dict["z"],
        "y_bar": result_dict["b"],
    })
    return df

```

```

[896]: df_windows = generate_windows_all_sessions(df_trimmedSessions, window_size=100,
    ↪step_size=50)
df_windows

```

Generated 112685 windows from 499 sessions.
(112685, 10)

```

[896]:
      user      activity      session_id  label  \
0    vbagdi  Stationary  vbagdi_Stationary_session1    1
1    vbagdi  Stationary  vbagdi_Stationary_session1    1
2    vbagdi  Stationary  vbagdi_Stationary_session1    1
3    vbagdi  Stationary  vbagdi_Stationary_session1    1
4    vbagdi  Stationary  vbagdi_Stationary_session1    1
...     ...         ...         ...         ...
112680 xichen  Elevator-down  xichen_Elevator-down_session2    7
112681 xichen  Elevator-down  xichen_Elevator-down_session2    7
112682 xichen  Elevator-down  xichen_Elevator-down_session2    7
112683 xichen  Elevator-down  xichen_Elevator-down_session2    7
112684 xichen  Elevator-down  xichen_Elevator-down_session2    7

      ts_start      ts_end  \
0    1.761522e+12  1.761522e+12
1    1.761522e+12  1.761522e+12
2    1.761522e+12  1.761522e+12
3    1.761522e+12  1.761522e+12
4    1.761522e+12  1.761522e+12

```

```

...
112680  1.760986e+12  1.760986e+12
112681  1.760986e+12  1.760986e+12
112682  1.760986e+12  1.760986e+12
112683  1.760986e+12  1.760986e+12
112684  1.760986e+12  1.760986e+12

                                accel_x  \
0      [-0.41012898, -0.42442918, -0.4482537, -0.4387...
1      [-0.49138388, -0.45327488, -0.46757117, -0.453...
2      [-0.43917343, -0.4963468, -0.4344172, -0.44394...
3      [-0.46320048, -0.45844033, -0.43462363, -0.425...
4      [-0.51043534, -0.5104379, -0.48185563, -0.4913...
...
112680  [2.641078, 2.6464617, 2.6799614, 2.7038896, 2...
112681  [5.5262237, 6.1944194, 6.1770716, 5.624928, 4...
112682  [2.5674987, 2.772085, 3.0921252, 2.7625136, 2...
112683  [2.7511477, 2.7469604, 2.742773, 2.7332017, 2...
112684  [2.2827528, 2.3042881, 2.3916261, 2.400001, 2...

                                accel_y  \
0      [-0.59539276, -0.6192039, -0.59539396, -0.5477...
1      [-0.5620831, -0.5858938, -0.571608, -0.6097048...
2      [-0.62877035, -0.5668639, -0.5621022, -0.55257...
3      [-0.6145036, -0.62402815, -0.51926285, -0.5716...
4      [-0.5430344, -0.57160705, -0.5668452, -0.53351...
...
112680  [-9.415757, -9.410972, -9.387043, -9.387043, -...
112681  [-9.863813, -9.635299, -9.369097, -9.134601, -...
112682  [-9.714262, -9.058628, -7.9908314, -8.016554, ...
112683  [-9.809975, -9.785448, -9.812965, -9.807582, -...
112684  [-9.507282, -9.334401, -9.163314, -9.226126, -...

                                accel_z  \
0      [9.893749, 9.9602, 9.912735, 9.884255, 9.90798...
1      [9.9744215, 9.926955, 9.917462, 9.950687, 9.92...
2      [9.941181, 9.88897, 9.898462, 9.96966, 10.0028...
3      [9.874716, 9.903194, 9.926927, 9.960152, 9.912...
4      [9.888985, 9.888985, 9.92221, 9.893729, 9.9317...
...
112680  [-0.20279172, -0.13100705, -0.083748795, -0.02...
112681  [2.324029, 1.2903296, 0.401396, -0.36729828, -...
112682  [0.6628119, 1.1659029, 1.8005991, 1.6151553, 1...
112683  [-0.12023934, 0.19501504, 0.50069815, 0.681954...
112684  [-0.05383851, 0.012562319, -0.15553348, -0.346...

                                y_bar

```



```

0      [1001.991366052565, 1001.9914551359675, 1001.9...
1      [1001.9945016363172, 1001.9945955960554, 1001...
2      [1001.9990209678789, 1001.9991344709576, 1001...
3      [1002.0028448795376, 1002.002887665007, 1002.0...
4      [1002.0050782423136, 1002.0051123518765, 1002...
...
112680 [1004.4823946348376, 1004.482649566673, 1004.4...
112681 [1004.5333368729985, 1004.5343228713247, 1004...
112682 [1004.5673750930287, 1004.5677514785841, 1004...
112683 [1004.5872392553915, 1004.5892310460314, 1004...
112684 [1004.6694465663747, 1004.670675533284, 1004.6...

```

[112685 rows x 10 columns]

```

[924]: # Pick a random window
window_rand = df_windows.sample(1).iloc[0]

# Extract metadata
user_rand = window_rand["user"]
activity_rand = window_rand["activity"]
session_rand = window_rand["session_id"]

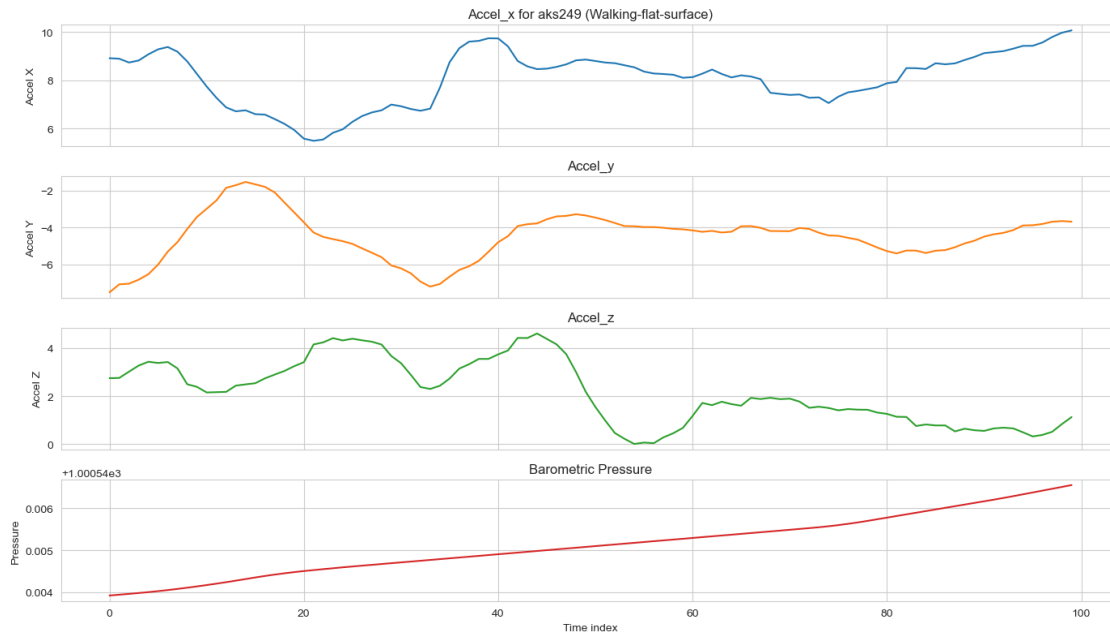
print(f" Random window from {user_rand} | {activity_rand} | {session_rand}")

# Rebuild it into a DataFrame with same structure as your plotting function
↳ expects
df_window = pd.DataFrame({
    "ts": np.linspace(window_rand["ts_start"], window_rand["ts_end"],
↳ len(window_rand["accel_x"])),
    "accel_x": window_rand["accel_x"],
    "accel_y": window_rand["accel_y"],
    "accel_z": window_rand["accel_z"],
    "y_bar": window_rand["y_bar"],
    "user": user_rand,
    "activity": activity_rand,
    "session_id": session_rand
})

# Plot using your existing function
plot_user_activity(df_window, user_id=user_rand, random=False)

```

Random window from aks249 | Walking-flat-surface | aks249_Walking-flat-surface_session2



7 Stationarity tests

```
[1195]: # Automatically compute lags from number of samples and sample rate
def choose_lags(n_samples, sample_rate=32, seconds=2):
    """Compute recommended number of lags based on sample count and time_
    horizon."""
    lags_boxjenkins = int(10 * np.log10(n_samples))
    lags_physical = int(seconds * sample_rate)
    return max(lags_boxjenkins, lags_physical)

# Visual method - choose p by ACF
def plot_acf(series, sig_name, lags=20):
    """Plot ACF to visually inspect AR order p."""
    fig, ax = plt.subplots(figsize=(8, 4))
    plot_acf(series, lags=lags, ax=ax)
    ax.set_title(f"{sig_name} ACF")
    plt.show()

# Pure Brockwell root condition
def brockwell_root_condition(phi):
    """
    Apply Brockwell & Davis (Ch.3) stationarity condition:
    Given AR coefficients ..._p, check if all roots of
    1 - z - ... - _p z^p = 0 satisfy |z| > 1.
    """
```

```

phi = np.asarray(phi, float)
coeffs = np.concatenate([[1], -phi])
roots = np.roots(coeffs)
moduli = np.abs(roots)
stationary = np.all(moduli > 1)

print(f" = {np.round(phi, 4)}")
for i, (r, m) in enumerate(zip(roots, moduli), 1):
    print(f" Root {i}: {r:.4f} | |z|={m:.4f}")
print(" Stationary" if stationary else " Non-stationary")

return {"phi": phi, "roots": roots, "moduli": moduli, "stationary": stationary}

# Estimate  $\hat{\phi}$  from data if not provided
def estimate_phi_from_data(series, p=2):
    """Estimate AR(p) coefficients from data using least squares."""
    series = np.asarray(series, float)
    model = AutoReg(series, lags=p).fit()
    phi = model.params[1:] # skip intercept
    print(f"Estimated  $\hat{\phi}$  = {np.round(phi, 4)}")
    return phi

# Main Function: Run Brockwell test per user/activity/signal
def arma_stationarity_user(df, activity_name="Walking-flat-surface",
                           user_id=None, random=True, p=2,
                           phi_dict=None, sample_rate=32, seconds=2):
    """
    Test for accel_x, accel_y, accel_z, and y_bar.
    - If phi_dict is provided, uses those values directly (no fitting).
    - Otherwise, estimates  $\hat{\phi}$  from data (least squares) for each signal.
    - Automatically determines lags from sample count and sampling rate.
    """
    # Pick user
    if random or user_id is None:
        users = df[df["activity"] == activity_name]["user"].unique()
        user_id = np.random.choice(users)
        print(f"Randomly selected user: {user_id}")
    else:
        print(f"Using specified user: {user_id}")

    results = {}
    for sig in ["accel_x", "accel_y", "accel_z", "y_bar"]:
        series = df[(df["user"] == user_id) &
                    (df["activity"] == activity_name)][sig].dropna()

        if len(series) < p + 5:

```

```

        print(f" Not enough data for {sig}")
        continue

    # Automatically choose number of lags
    lags = choose_lags(len(series), sample_rate=sample_rate,
↪seconds=seconds)
    print(f"ACF for {sig.upper()} | {lags} lags (auto)")

    # Plot ACF
    plot_acf_(series, sig_name=sig, lags=lags)

    # Estimate or use provided
    if phi_dict and sig in phi_dict:
        phi = phi_dict[sig]
        print(f"{sig.upper()} | Using provided = {phi} | User: {user_id}")
    else:
        print(f"{sig.upper()} | Estimating ^ | AR({p}) | User: {user_id}")
        phi = estimate_phi_from_data(series, p=p)

    # Brockwell root condition
    results[sig] = brockwell_root_condition(phi)

return results

```

7.1 Test one window of at a time

```

[1191]: def arma_stationarity_random_window(df_windows, p=2, sample_rate=32, seconds=2):
    """
    Pick a random window and use the existing arma_stationarity_user()
    to print estimates, ACFs, and full Brockwell root output.
    """
    # Randomly select one window
    win = df_windows.sample(1).iloc[0]
    activity = win["activity"]

    # Build temp df for arma_stationarity_user
    df_tmp = pd.DataFrame({
        "ts": np.linspace(win["ts_start"], win["ts_end"],
↪len(win["accel_x"])),
        "accel_x": win["accel_x"],
        "accel_y": win["accel_y"],
        "accel_z": win["accel_z"],
        "y_bar": win["y_bar"],
        "user": win["user"],
        "activity": activity
    })

```

```

    # Force arma_stationarity_user() to pick a random user from the same
    ↪activity
    print(f"Random window selected from activity: {activity}")
    results = arma_stationarity_user(
        df_tmp,
        activity_name=activity,
        user_id=None,    # forces random user selection + prints "Randomly
    ↪selected user"
        random=True,
        p=p,
        sample_rate=sample_rate,
        seconds=seconds
    )

    return results

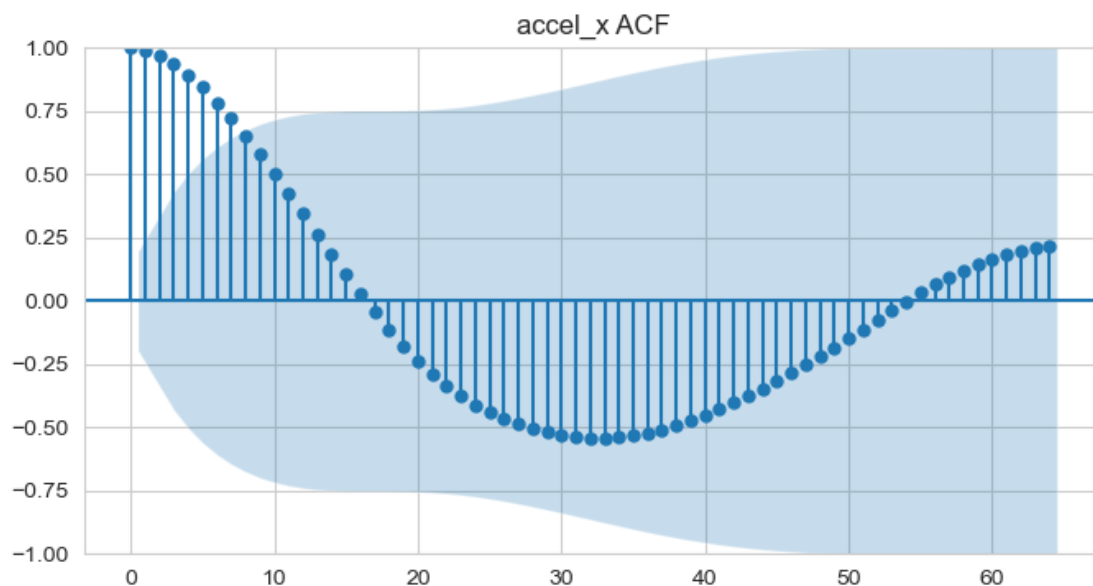
```

```

[1196]: results = arma_stationarity_random_window(df_windows, p=2)
results

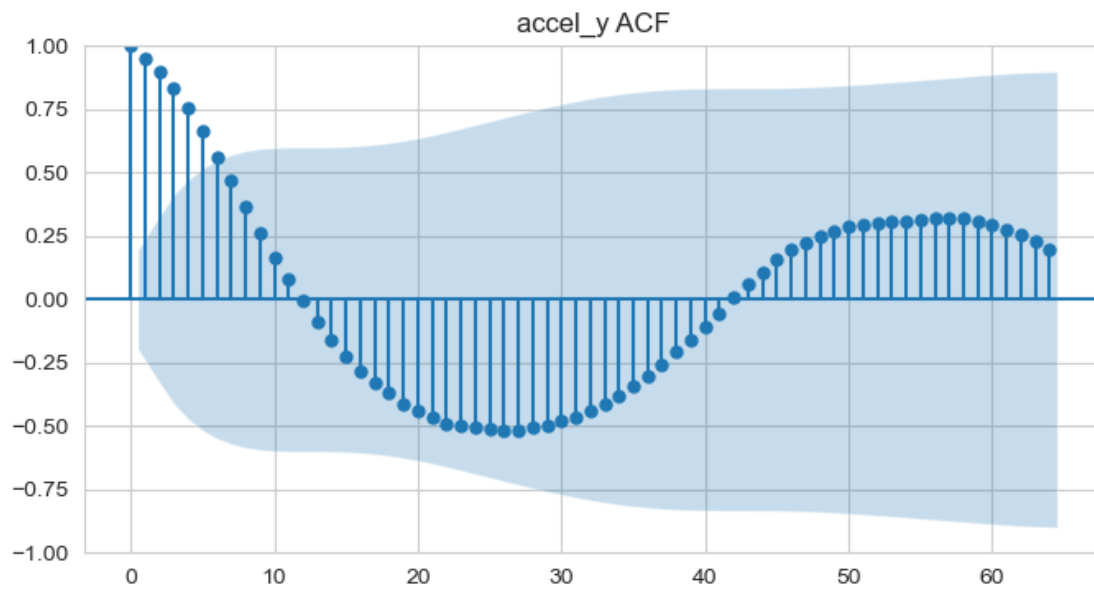
```

Random window selected from activity: Stationary
 Randomly selected user: Grack
 ACF for ACCEL_X | 64 lags (auto)

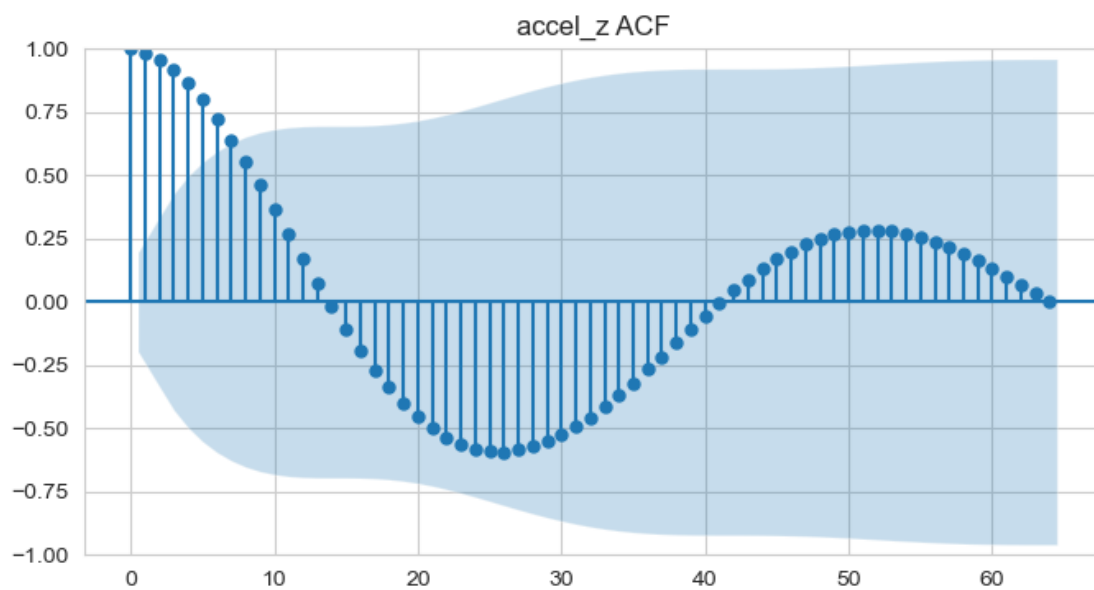


ACCEL_X | Estimating $\hat{\cdot}$ | AR(2) | User: Grack
 Estimated $\hat{\cdot}$ = [1.5096 -0.525]
 = [1.5096 -0.525]
 Root 1: 0.9662 | |z|=0.9662
 Root 2: 0.5434 | |z|=0.5434

Non-stationary
ACF for ACCEL_Y | 64 lags (auto)



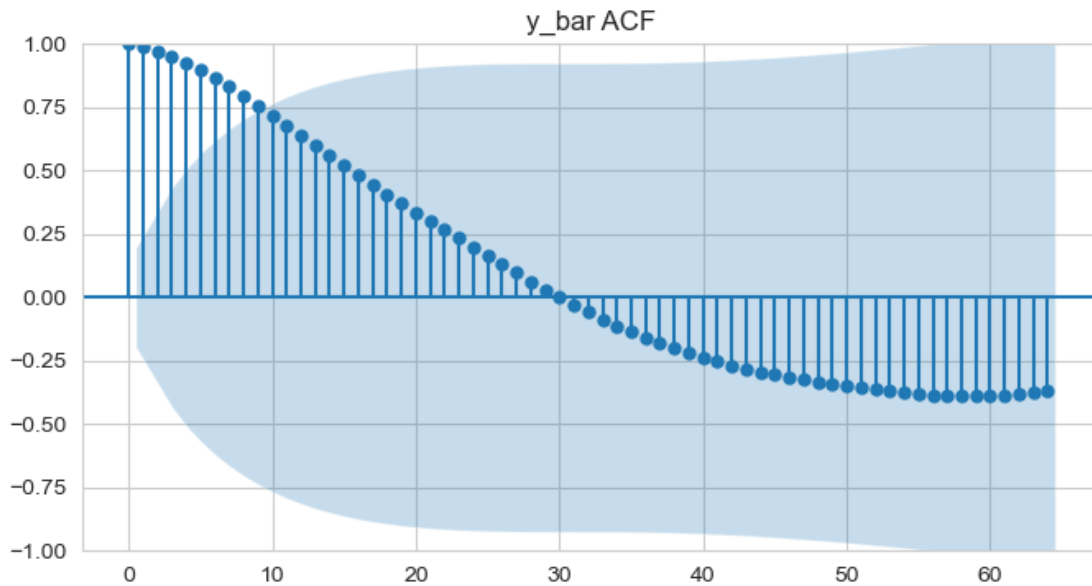
ACCEL_Y | Estimating $\hat{\rho}$ | AR(2) | User: Grack
Estimated $\hat{\rho} = [0.9805 \ -0.0343]$
= [0.9805 -0.0343]
Root 1: 0.9442 | $|z|=0.9442$
Root 2: 0.0363 | $|z|=0.0363$
Non-stationary
ACF for ACCEL_Z | 64 lags (auto)



```

ACCEL_Z | Estimating ^ | AR(2) | User: Grack
Estimated ^ = [ 1.7209 -0.7375]
           = [ 1.7209 -0.7375]
Root 1: 0.9138 | |z|=0.9138
Root 2: 0.8072 | |z|=0.8072
Non-stationary
ACF for Y_BAR | 64 lags (auto)

```



```

Y_BAR | Estimating ^ | AR(2) | User: Grack
Estimated ^ = [ 1.9784 -0.983 ]
           = [ 1.9784 -0.983 ]
Root 1: 0.9892+0.0671j | |z|=0.9915
Root 2: 0.9892-0.0671j | |z|=0.9915
Non-stationary

```

```

[1196]: {'accel_x': {'phi': array([ 1.50959749, -0.52504295]),
  'roots': array([0.96617091, 0.54342658]),
  'moduli': array([0.96617091, 0.54342658]),
  'stationary': False},
'accel_y': {'phi': array([ 0.98050586, -0.03428642]),
  'roots': array([0.94419292, 0.03631294]),
  'moduli': array([0.94419292, 0.03631294]),
  'stationary': False},
'accel_z': {'phi': array([ 1.72091783, -0.73754805]),
  'roots': array([0.91376456, 0.80715327]),

```

```

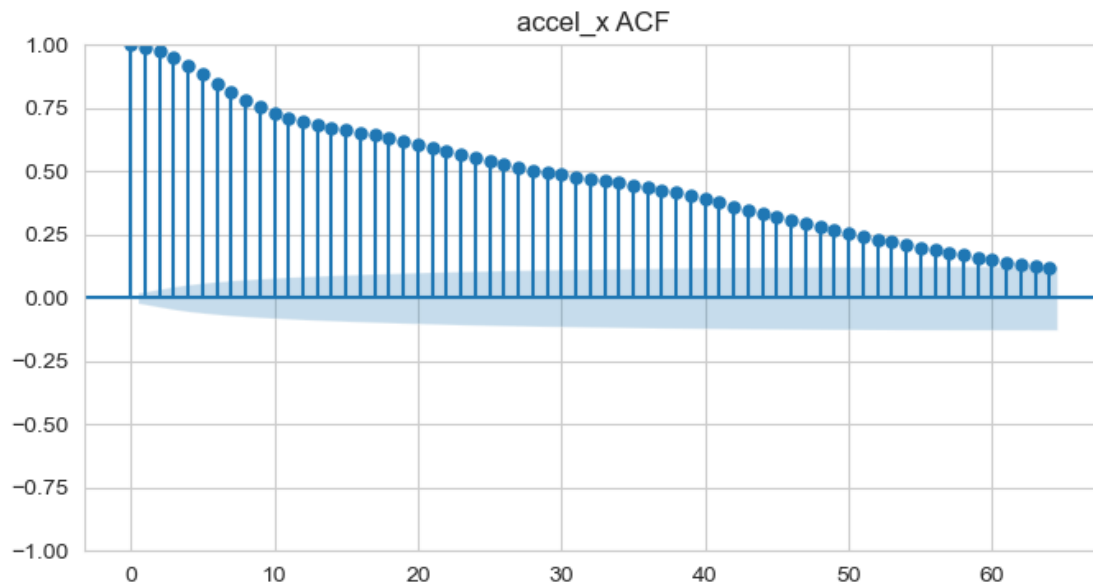
'moduli': array([0.91376456, 0.80715327]),
'stationary': False},
'y_bar': {'phi': array([ 1.97836117, -0.9829755 ]),
'roots': array([0.98918059+0.06706167j, 0.98918059-0.06706167j]),
'moduli': array([0.99145121, 0.99145121]),
'stationary': False}}

```

```
[1197]: arma_stationarity_user(df_allMeansTrimmed, activity_walk, random=True)
```

Randomly selected user: nbw32

ACF for ACCEL_X | 64 lags (auto)



ACCEL_X | Estimating $\hat{\cdot}$ | AR(2) | User: nbw32

Estimated $\hat{\cdot}$ = [1.5191 -0.5325]

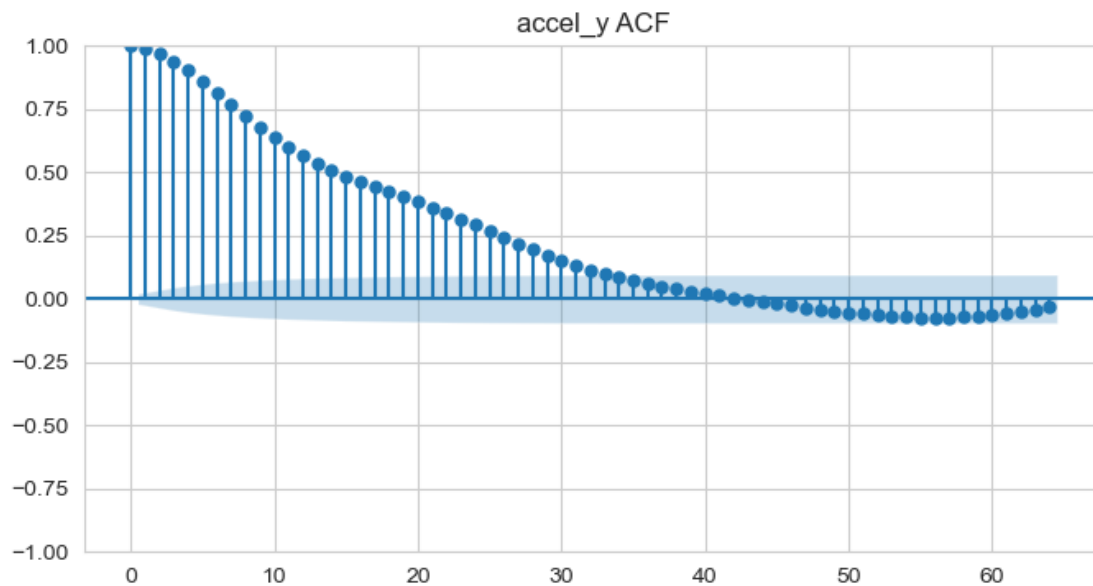
= [1.5191 -0.5325]

Root 1: 0.9704 | $|z|=0.9704$

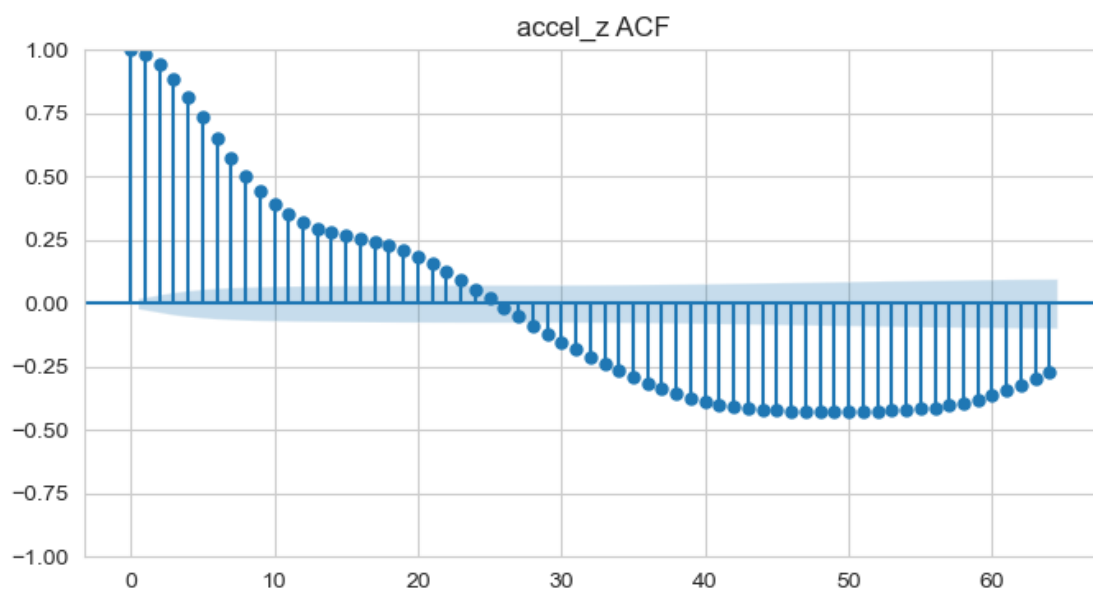
Root 2: 0.5488 | $|z|=0.5488$

Non-stationary

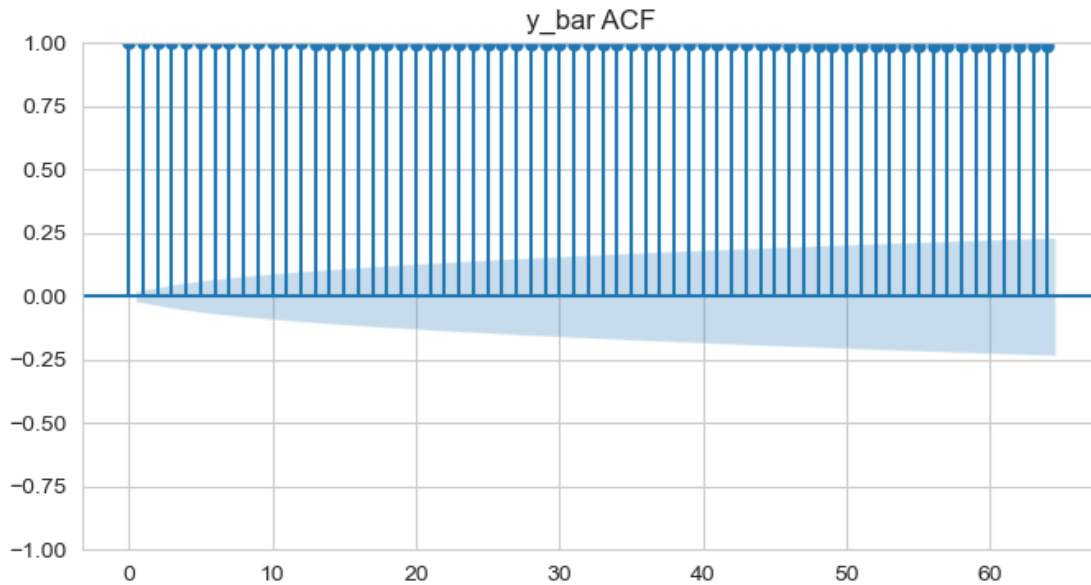
ACF for ACCEL_Y | 64 lags (auto)



```
ACCEL_Y | Estimating ^ | AR(2) | User: nbw32
Estimated ^ = [ 1.6177 -0.6324]
= [ 1.6177 -0.6324]
Root 1: 0.9568 | |z|=0.9568
Root 2: 0.6609 | |z|=0.6609
Non-stationary
ACF for ACCEL_Z | 64 lags (auto)
```



ACCEL_Z | Estimating $\hat{\cdot}$ | AR(2) | User: nbw32
 Estimated $\hat{\cdot}$ = [1.5665 -0.5958]
 = [1.5665 -0.5958]
 Root 1: 0.9163 | $|z|=0.9163$
 Root 2: 0.6502 | $|z|=0.6502$
 Non-stationary
 ACF for Y_BAR | 64 lags (auto)



Y_BAR | Estimating $\hat{\cdot}$ | AR(2) | User: nbw32
 Estimated $\hat{\cdot}$ = [1.9989 -0.9989]
 = [1.9989 -0.9989]
 Root 1: 0.9995+0.0019j | $|z|=0.9995$
 Root 2: 0.9995-0.0019j | $|z|=0.9995$
 Non-stationary

```
[1197]: {'accel_x': {'phi': array([ 1.51913958, -0.53251523]),
  'roots': array([0.97035651, 0.54878307]),
  'moduli': array([0.97035651, 0.54878307]),
  'stationary': False},
'accel_y': {'phi': array([ 1.61773285, -0.63237263]),
  'roots': array([0.95682679, 0.66090606]),
  'moduli': array([0.95682679, 0.66090606]),
  'stationary': False},
'accel_z': {'phi': array([ 1.5664928 , -0.59577954]),
  'roots': array([0.91627039, 0.65022241]),
  'moduli': array([0.91627039, 0.65022241]),
  'stationary': False},
'y_bar': {'phi': array([ 1.99892307, -0.99892685]),
```

```
'roots': array([0.99946153+0.0018698j, 0.99946153-0.0018698j]),
'moduli': array([0.99946328, 0.99946328]),
'stationary': False}}
```

8 Test removing seasonality

```
[ ]: # Seasonal lag estimator from ACF
def estimate_seasonal_lag_acf(x, sample_rate=32, min_sec=0.6, max_sec=1.2,
    ↪min_prom=0.1):
    """
    Estimate seasonal lag s from the first prominent ACF peak in [min_sec,
    ↪max_sec].
    Returns None if no clear seasonal peak.
    """
    x = np.asarray(x, float)
    if len(x) < int(max_sec*sample_rate)+5:
        return None
    # unbiased=False keeps behavior fast and stable for windowed data
    max_lag = int(max_sec * sample_rate)
    ac = acf(x, nlags=max_lag, fft=True)
    # search range
    lo = max(1, int(min_sec * sample_rate))
    hi = max(2, int(max_sec * sample_rate))
    seg = ac[lo:hi+1]
    if seg.size == 0:
        return None
    k = lo + np.argmax(seg)
    peak = ac[k]
    # simple prominence check against local baseline (lag 1)
    baseline = ac[1] if len(ac) > 2 else 0.0
    if peak - baseline < min_prom:
        return None
    return int(k)

# Seasonal ordinary differencing on one 1D array
def seasonal_diff_array(x, s=None, ordinary_order=0):
    y = np.asarray(x, float)
    if s is not None and s >= 1 and s < len(y):
        y = y[s:] - y[:-s]
    if ordinary_order and len(y) > ordinary_order:
        y = np.diff(y, n=ordinary_order)
    return y

# Process a single window row (arrays in columns)
def process_window_row(row, sample_rate=32, s_map=None, # optional dict like
    ↪{"Walking-flat-surface": 32})
```

```

        min_sec=0.6, max_sec=1.2, min_prom=0.1,
        ordinary_order=0, estimate_if_missing=True):
    """
    Apply  $(1 - B^s)$  then  $(1 - B)^d$  to accel_x/y/z/y_bar for one row.
    s chosen by:
        1) s_map[activity] if provided,
        2) ACF estimate per-signal (use accel_x as primary) if
    ↪estimate_if_missing,
        3) None (skip seasonal) otherwise.
    Returns (processed_row_dict, s_used)
    """
    activity = row["activity"]
    # prefer a fixed s by activity if supplied
    s_used = s_map.get(activity) if (s_map is not None and activity in s_map)
    ↪else None

    # if not supplied, estimate from accel_x (most gait-dominant)
    if s_used is None and estimate_if_missing:
        s_used = estimate_seasonal_lag_acf(row["accel_x"], sample_rate,
    ↪min_sec, max_sec, min_prom)

    out = dict(row)
    for col in ["accel_x", "accel_y", "accel_z", "y_bar"]:
        out[col] = seasonal_diff_array(out[col], s=s_used,
    ↪ordinary_order=ordinary_order)
    out["s_used"] = s_used
    return out

# Ray remote worker
@ray.remote
def process_window_row_remote(row_dict, sample_rate=32, s_map=None,
                              min_sec=0.6, max_sec=1.2, min_prom=0.1,
                              ordinary_order=0, estimate_if_missing=True):
    return process_window_row(row_dict, sample_rate, s_map, min_sec, max_sec,
    ↪min_prom,
                              ordinary_order, estimate_if_missing)

# Apply to an entire windows DataFrame in parallel
def seasonal_difference_all_windows(df_windows, sample_rate=32,
                                    s_map=None, # e.g.,
    ↪{"Walking-flat-surface": 32}

                                    min_sec=0.6, max_sec=1.2, min_prom=0.1,
                                    ordinary_order=0, # 0, 1, or 2; do
    ↪seasonal first

                                    estimate_if_missing=True):
    """

```

```

df_windows columns expected:
    user, activity, session_id, label, ts_start, ts_end,
    accel_x, accel_y, accel_z, y_bar (arrays per row)
Returns a new DataFrame with arrays seasonally differenced (and then
↳ordinary differencing).
Adds s_used per row.
"""
if not ray.is_initialized():
    ray.init(ignore_reinit_error=True)

# Convert each row to dict for Ray
rows = [r._asdict() if hasattr(r, "_asdict") else r.to_dict() for _, r in
↳df_windows.iterrows()]
futures = [
    process_window_row_remote.remote(
        row, sample_rate, s_map, min_sec, max_sec, min_prom,
↳ordinary_order, estimate_if_missing
    )
    for row in rows
]
processed = ray.get(futures)

# Drop rows where arrays got too short after differencing
keep = []
for r in processed:
    ax = np.asarray(r["accel_x"])
    ay = np.asarray(r["accel_y"])
    az = np.asarray(r["accel_z"])
    br = np.asarray(r["y_bar"])
    if min(len(ax), len(ay), len(az), len(br)) >= 8: # keep short but
↳usable windows
        keep.append(r)
if not keep:
    return pd.DataFrame(columns=df_windows.columns.tolist() + ["s_used"])

return pd.DataFrame(keep)

```

```

[ ]: df_windows_seas = seasonal_difference_all_windows(
    df_windows,
    sample_rate=32,
    s_map=None, # no fixed seasonality -
↳auto-estimate per window
    min_sec=0.6,
    max_sec=1.2,
    min_prom=0.1,
    ordinary_order=0, # only seasonal differencing
↳(you can later do +1 for regular differencing)
)

```

```

        estimate_if_missing=True
    )
    df_windows_seas

```

```

[ ]:
      user      activity      session_id  label \
0      vbagdi      Stationary      vbagdi_Stationary_session1      1
1      vbagdi      Stationary      vbagdi_Stationary_session1      1
2      vbagdi      Stationary      vbagdi_Stationary_session1      1
3      vbagdi      Stationary      vbagdi_Stationary_session1      1
4      vbagdi      Stationary      vbagdi_Stationary_session1      1
...
112680 xichen      Elevator-down      xichen_Elevator-down_session2      7
112681 xichen      Elevator-down      xichen_Elevator-down_session2      7
112682 xichen      Elevator-down      xichen_Elevator-down_session2      7
112683 xichen      Elevator-down      xichen_Elevator-down_session2      7
112684 xichen      Elevator-down      xichen_Elevator-down_session2      7

      ts_start      ts_end \
0      1.761522e+12      1.761522e+12
1      1.761522e+12      1.761522e+12
2      1.761522e+12      1.761522e+12
3      1.761522e+12      1.761522e+12
4      1.761522e+12      1.761522e+12
...
112680 1.760986e+12      1.760986e+12
112681 1.760986e+12      1.760986e+12
112682 1.760986e+12      1.760986e+12
112683 1.760986e+12      1.760986e+12
112684 1.760986e+12      1.760986e+12

      accel_x \
0      [-0.41012898, -0.42442918, -0.4482537, -0.4387...
1      [-0.49138388, -0.45327488, -0.46757117, -0.453...
2      [-0.43917343, -0.4963468, -0.4344172, -0.44394...
3      [-0.46320048, -0.45844033, -0.43462363, -0.425...
4      [-0.51043534, -0.5104379, -0.48185563, -0.4913...
...
112680 [2.641078, 2.6464617, 2.6799614, 2.7038896, 2...
112681 [5.5262237, 6.1944194, 6.1770716, 5.624928, 4...
112682 [2.5674987, 2.772085, 3.0921252, 2.7625136, 2...
112683 [2.7511477, 2.7469604, 2.742773, 2.7332017, 2...
112684 [2.2827528, 2.3042881, 2.3916261, 2.400001, 2...

      accel_y \
0      [-0.59539276, -0.6192039, -0.59539396, -0.5477...
1      [-0.5620831, -0.5858938, -0.571608, -0.6097048...
2      [-0.62877035, -0.5668639, -0.5621022, -0.55257...

```

```

3      [-0.6145036, -0.62402815, -0.51926285, -0.5716...
4      [-0.5430344, -0.57160705, -0.5668452, -0.53351...
...
112680 [-9.415757, -9.410972, -9.387043, -9.387043, -...
112681 [-9.863813, -9.635299, -9.369097, -9.134601, -...
112682 [-9.714262, -9.058628, -7.9908314, -8.016554, ...
112683 [-9.809975, -9.785448, -9.812965, -9.807582, -...
112684 [-9.507282, -9.334401, -9.163314, -9.226126, -...

```

```

                                accel_z \
0      [9.893749, 9.9602, 9.912735, 9.884255, 9.90798...
1      [9.9744215, 9.926955, 9.917462, 9.950687, 9.92...
2      [9.941181, 9.88897, 9.898462, 9.96966, 10.0028...
3      [9.874716, 9.903194, 9.926927, 9.960152, 9.912...
4      [9.888985, 9.888985, 9.92221, 9.893729, 9.9317...
...
112680 [-0.20279172, -0.13100705, -0.083748795, -0.02...
112681 [2.324029, 1.2903296, 0.401396, -0.36729828, -...
112682 [0.6628119, 1.1659029, 1.8005991, 1.6151553, 1...
112683 [-0.12023934, 0.19501504, 0.50069815, 0.681954...
112684 [-0.05383851, 0.012562319, -0.15553348, -0.346...

```

```

                                y_bar  s_used
0      [1001.991366052565, 1001.9914551359675, 1001.9...   NaN
1      [1001.9945016363172, 1001.9945955960554, 1001...   NaN
2      [1001.9990209678789, 1001.9991344709576, 1001...   NaN
3      [1002.0028448795376, 1002.002887665007, 1002.0...   NaN
4      [1002.0050782423136, 1002.0051123518765, 1002...   NaN
...
112680 [1004.4823946348376, 1004.482649566673, 1004.4...   NaN
112681 [1004.5333368729985, 1004.5343228713247, 1004...   NaN
112682 [1004.5673750930287, 1004.5677514785841, 1004...   NaN
112683 [1004.5872392553915, 1004.5892310460314, 1004...   NaN
112684 [1004.6694465663747, 1004.670675533284, 1004.6...   NaN

```

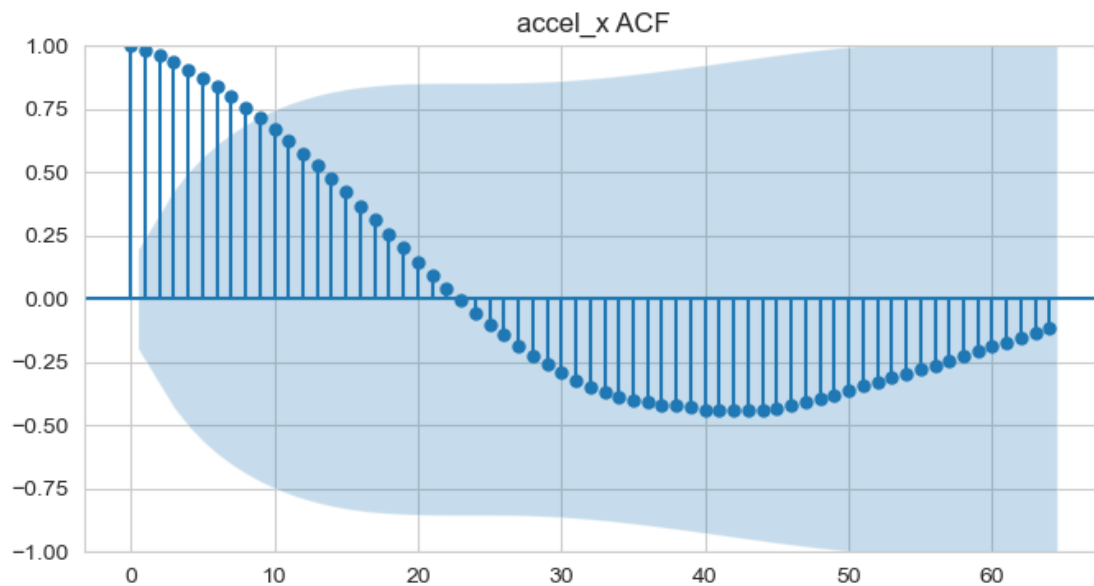
```
[112685 rows x 11 columns]
```

```
[1198]: results = arma_stationarity_random_window(df_windows_seas, p=2)
results
```

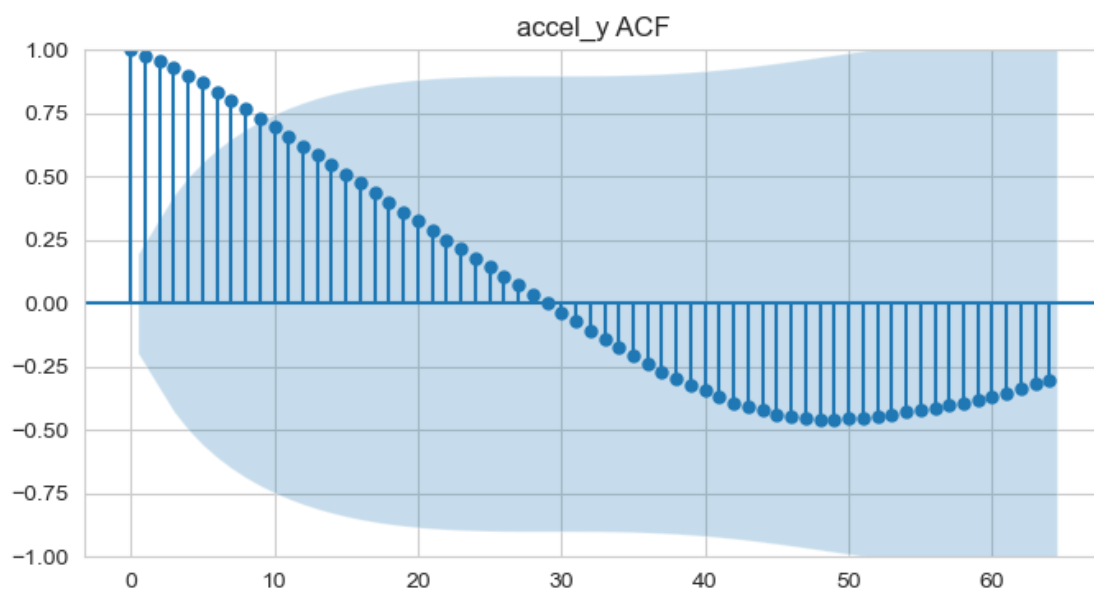
```

Random window selected from activity: Walking-down-stairs
Randomly selected user: xichen
ACF for ACCEL_X | 64 lags (auto)

```



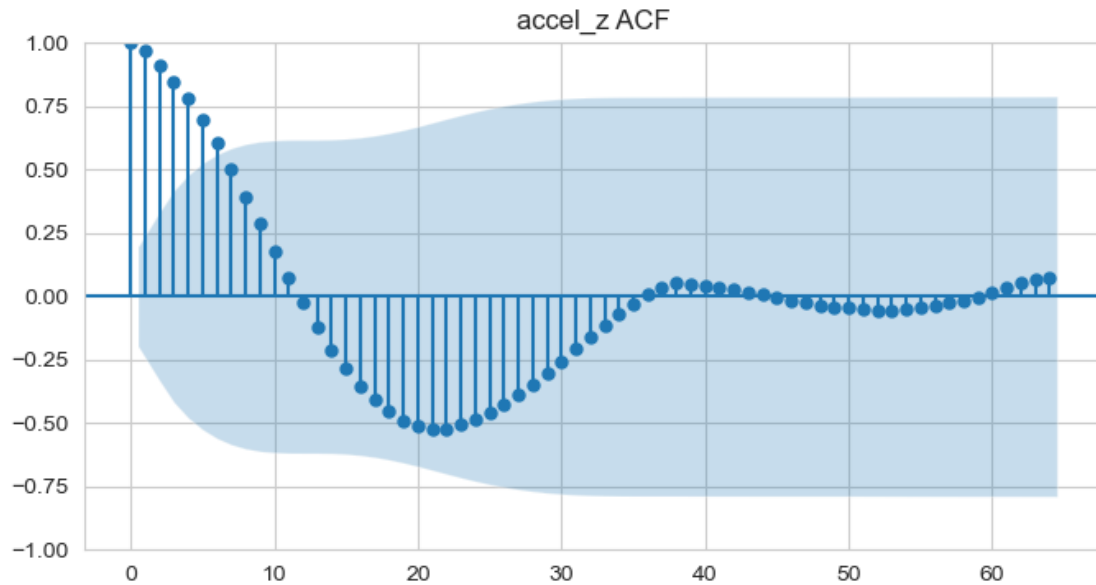
```
ACCEL_X | Estimating ^ | AR(2) | User: xichen
Estimated ^ = [ 1.565 -0.5651]
= [ 1.565 -0.5651]
Root 1: 0.9996 | |z|=0.9996
Root 2: 0.5653 | |z|=0.5653
Non-stationary
ACF for ACCEL_Y | 64 lags (auto)
```




```

ACCEL_Y | Estimating ^ | AR(2) | User: xichen
Estimated ^ = [ 1.3848 -0.3776]
            = [ 1.3848 -0.3776]
            Root 1: 1.0114 | |z|=1.0114
            Root 2: 0.3733 | |z|=0.3733
            Non-stationary
ACF for ACCEL_Z | 64 lags (auto)

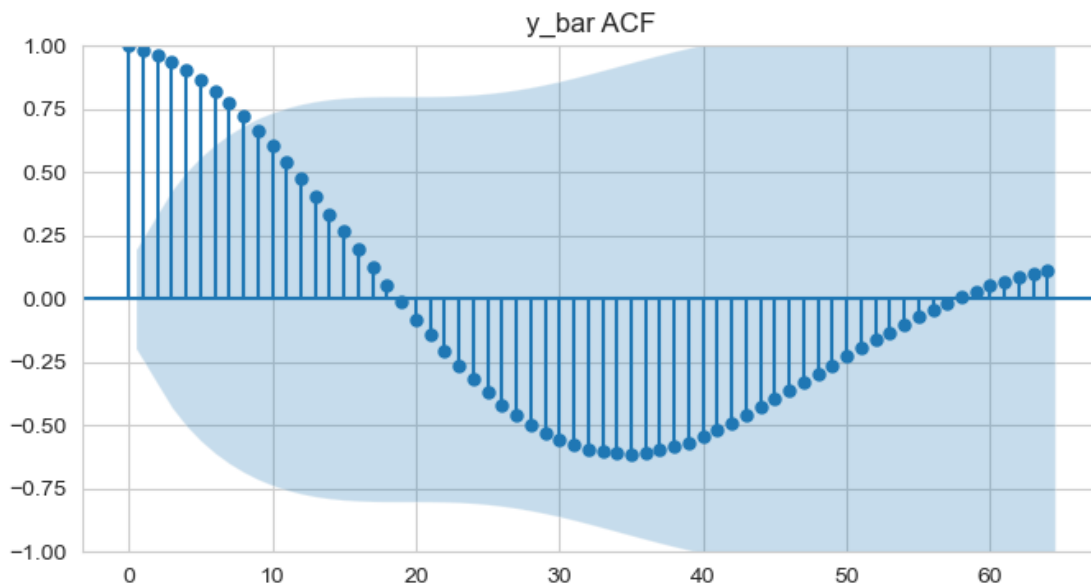
```



```

ACCEL_Z | Estimating ^ | AR(2) | User: xichen
Estimated ^ = [ 1.579 -0.6138]
            = [ 1.579 -0.6138]
            Root 1: 0.8872 | |z|=0.8872
            Root 2: 0.6919 | |z|=0.6919
            Non-stationary
ACF for Y_BAR | 64 lags (auto)

```



Y_BAR | Estimating $\hat{\cdot}$ | AR(2) | User: xichen

Estimated $\hat{\cdot}$ = [2.0018 -1.0082]

= [2.0018 -1.0082]

Root 1: 1.0009+0.0797j | |z|=1.0041

Root 2: 1.0009-0.0797j | |z|=1.0041

Stationary

```
[1198]: {'accel_x': {'phi': array([ 1.56497916, -0.56513725]),
  'roots': array([0.99963628, 0.56534287]),
  'moduli': array([0.99963628, 0.56534287]),
  'stationary': False},
  'accel_y': {'phi': array([ 1.38475979, -0.37760855]),
  'roots': array([1.01141182, 0.37334797]),
  'moduli': array([1.01141182, 0.37334797]),
  'stationary': False},
  'accel_z': {'phi': array([ 1.57902383, -0.61379475]),
  'roots': array([0.8871557 , 0.69186813]),
  'moduli': array([0.8871557 , 0.69186813]),
  'stationary': False},
  'y_bar': {'phi': array([ 2.0018389 , -1.00818513]),
  'roots': array([1.00091945+0.07965793j, 1.00091945-0.07965793j]),
  'moduli': array([1.00408422, 1.00408422]),
  'stationary': True}}
```

9 Test with Differencing

```
[ ]: # Difference one window (row-wise on arrays) ---
def difference_window_arrays(row, order=1):
    """Apply n-th order differencing to the array columns in one window row."""
    diffed = {}
    for col in ["accel_x", "accel_y", "accel_z", "y_bar"]:
        arr = np.asarray(row[col])
        if arr.ndim == 1 and len(arr) > order:
            diffed[col] = np.diff(arr, n=order)
        else:
            diffed[col] = np.array([])
    diffed["user"] = row["user"]
    diffed["activity"] = row["activity"]
    diffed["session_id"] = row["session_id"]
    diffed["label"] = row["label"]
    diffed["ts_start"] = row["ts_start"]
    diffed["ts_end"] = row["ts_end"]
    return diffed

# Ray remote worker ---
@ray.remote
def difference_window_arrays_remote(row, order=1):
    return difference_window_arrays(row, order)

#Parallel differencing across all windows ---
def difference_all_windows(df_windows, order=1):
    """
    Parallel differencing for array-style windowed data.
    Each window row is processed independently.
    """
    if not ray.is_initialized():
        ray.init(ignore_reinit_error=True)

    rows = df_windows.to_dict("records")
    futures = [difference_window_arrays_remote.remote(row, order=order) for row in rows]
    results = ray.get(futures)
    valid = [r for r in results if all(len(r[c]) > 0 for c in ["accel_x", "accel_y", "accel_z", "y_bar"])]

    df_diff = pd.DataFrame(valid)
    print(f"Differenced {len(df_diff)} windows | order={order}")
    return df_diff
```

```
[1044]: # First difference
df_diff1 = difference_all_windows(df_windows_seas, order=1)
df_diff1
```

Differenced 112685 windows | order=1

```
[1044]:
0      [-0.014300200000000004, -0.023824519999999996, 0...
1      [0.038109000000000004, -0.014296289999999999, 0...
2      [-0.05717337, 0.061929599999999974, -0.0095321...
3      [0.0047601500000000185, 0.0238167000000000024, ...
4      [-2.559999999847967e-06, 0.028582270000000002,...
...
112680 [0.0053837000000000324, 0.033499699999999969, 0...
112681 [0.66819570000000001, -0.017347800000000047, -0...
112682 [0.204586300000000033, 0.32004019999999998, -0.3...
112683 [-0.0041873000000000338, -0.00418739999999973, ...
112684 [0.0215353000000000035, 0.08733799999999992, 0...

                                accel_y \
0      [-0.0238111400000000064, 0.023809940000000003, 0...
1      [-0.0238107000000000073, 0.0142858000000000015, ...
2      [0.0619064500000000006, 0.004761699999999998, 0.0...
3      [-0.009524549999999965, 0.10476529999999995, -...
4      [-0.028572649999999977, 0.004761849999999956, ...
...
112680 [0.0047850000000000039, 0.023928999999999898, 0...
112681 [0.228514000000000055, 0.26620199999999998, 0.23...
112682 [0.65563399999999992, 1.06779660000000003, -0.02...
112683 [0.0245269999999999077, -0.027516999999999957, 0...
112684 [0.172881000000000028, 0.171087, -0.06281200000...

                                accel_z \
0      [0.066451000000000007, -0.0474650000000000076, -0...
1      [-0.0474665000000000052, -0.0094929999999999086, ...
2      [-0.0522109999999999785, 0.0094919999999999834, ...
3      [0.028477999999999978, 0.0237330000000000004, 0...
4      [0.0, 0.033224999999999984, -0.02848099999999993...
...
112680 [0.07178467, 0.047258255000000001, 0.057427745,...
112681 [-1.0336994, -0.8889336, -0.76869428, -0.54795...
112682 [0.503091000000000001, 0.63469619999999998, -0.18...
112683 [0.31525438, 0.30568310999999999, 0.18125629000...
112684 [0.066400829, -0.168095799, -0.19082759, -0.01...

                                y_bar      user \
0      [8.908340248581226e-05, 8.957698514677759e-05,...  vbagdi
1      [9.395973825121473e-05, 9.6837415412665e-05, 9...  vbagdi
```

```

2      [0.00011350307875090948, 0.0001092897730359254... vbagdi
3      [4.2785469418049615e-05, 3.9055244542396395e-0... vbagdi
4      [3.4109562875528354e-05, 4.301715500787395e-05... vbagdi
...
112680 [0.0002549318353430863, 0.00030409503210648836... xichen
112681 [0.000985998326200388, 0.0009816302371064012, ... xichen
112682 [0.00037638555545527197, 0.0003303049004443892... xichen
112683 [0.0019917906398632113, 0.002070664832217517, ... xichen
112684 [0.0012289669092524491, 0.001255246469213489, ... xichen

```

	activity	session_id	label	ts_start \
0	Stationary	vbagdi_Stationary_session1	1	1.761522e+12
1	Stationary	vbagdi_Stationary_session1	1	1.761522e+12
2	Stationary	vbagdi_Stationary_session1	1	1.761522e+12
3	Stationary	vbagdi_Stationary_session1	1	1.761522e+12
4	Stationary	vbagdi_Stationary_session1	1	1.761522e+12
...	...	...	...	...
112680	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12
112681	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12
112682	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12
112683	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12
112684	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12

	ts_end
0	1.761522e+12
1	1.761522e+12
2	1.761522e+12
3	1.761522e+12
4	1.761522e+12
...	...
112680	1.760986e+12
112681	1.760986e+12
112682	1.760986e+12
112683	1.760986e+12
112684	1.760986e+12

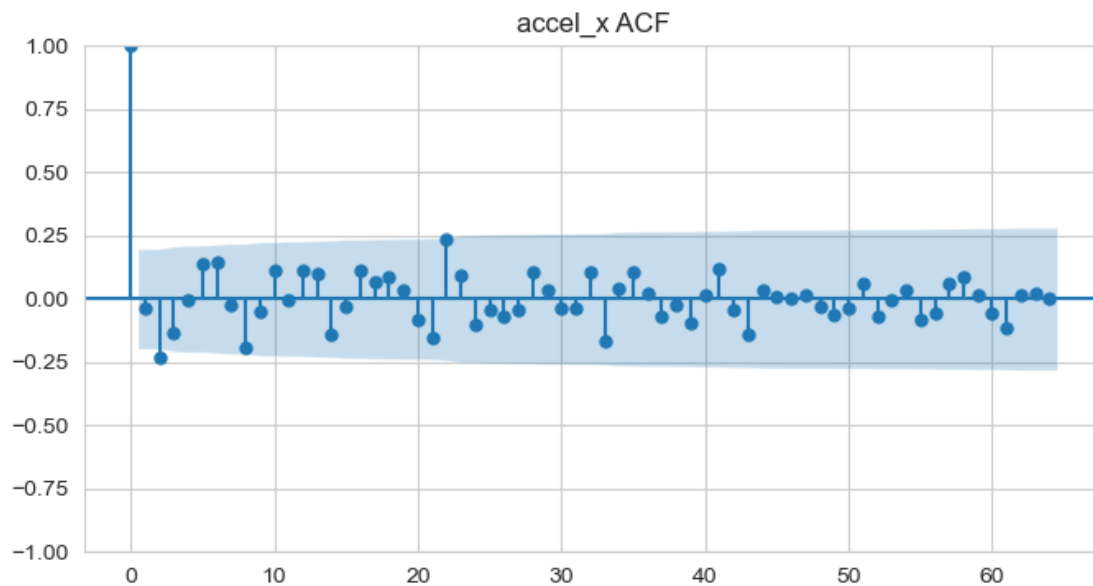
[112685 rows x 10 columns]

```
[1199]: results = arma_stationarity_random_window(df_diff1, p=2)
results
```

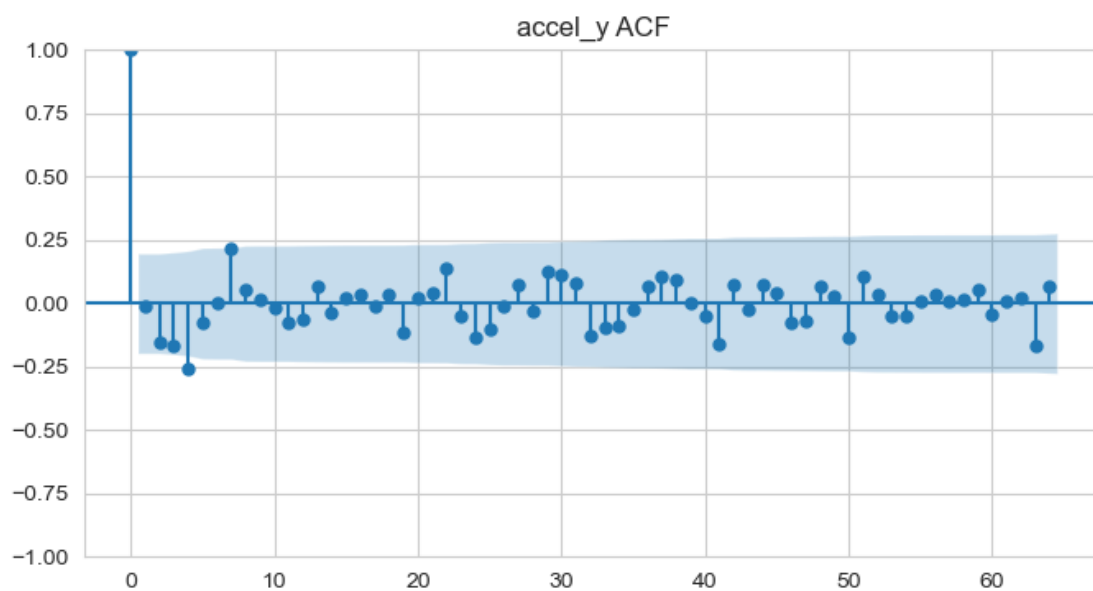
Random window selected from activity: Stationary

Randomly selected user: wh298

ACF for ACCEL_X | 64 lags (auto)



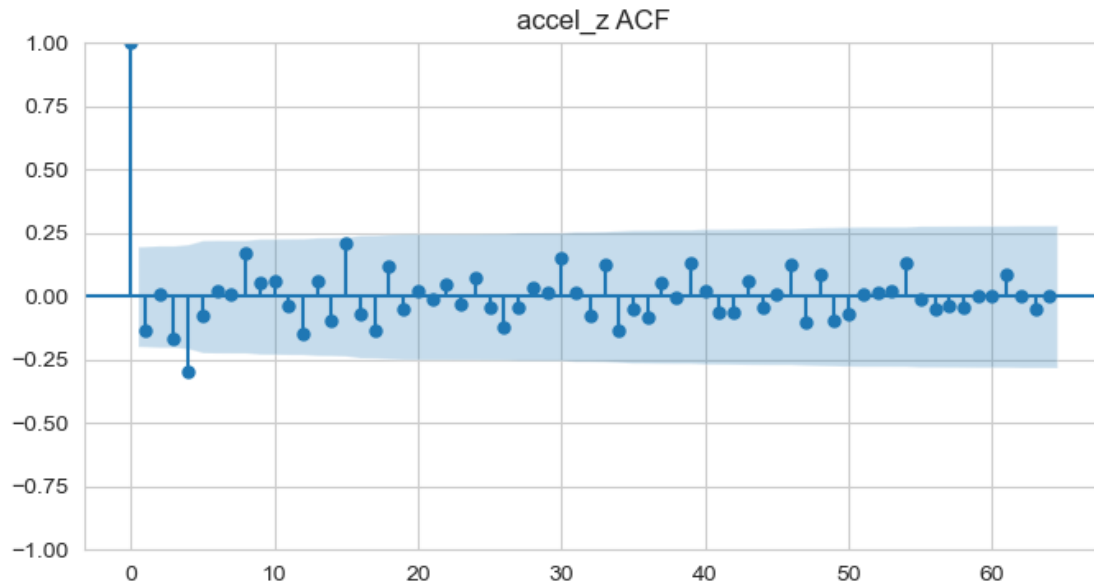
```
ACCEL_X | Estimating ^ | AR(2) | User: wh298
Estimated ^ = [-0.0511 -0.2384]
= [-0.0511 -0.2384]
Root 1: -0.0256+0.4876j | |z|=0.4883
Root 2: -0.0256-0.4876j | |z|=0.4883
Non-stationary
ACF for ACCEL_Y | 64 lags (auto)
```



```

ACCEL_Y | Estimating ^ | AR(2) | User: wh298
Estimated ^ = [-0.0123 -0.1546]
           = [-0.0123 -0.1546]
Root 1: -0.0062+0.3931j | |z|=0.3931
Root 2: -0.0062-0.3931j | |z|=0.3931
Non-stationary
ACF for ACCEL_Z | 64 lags (auto)

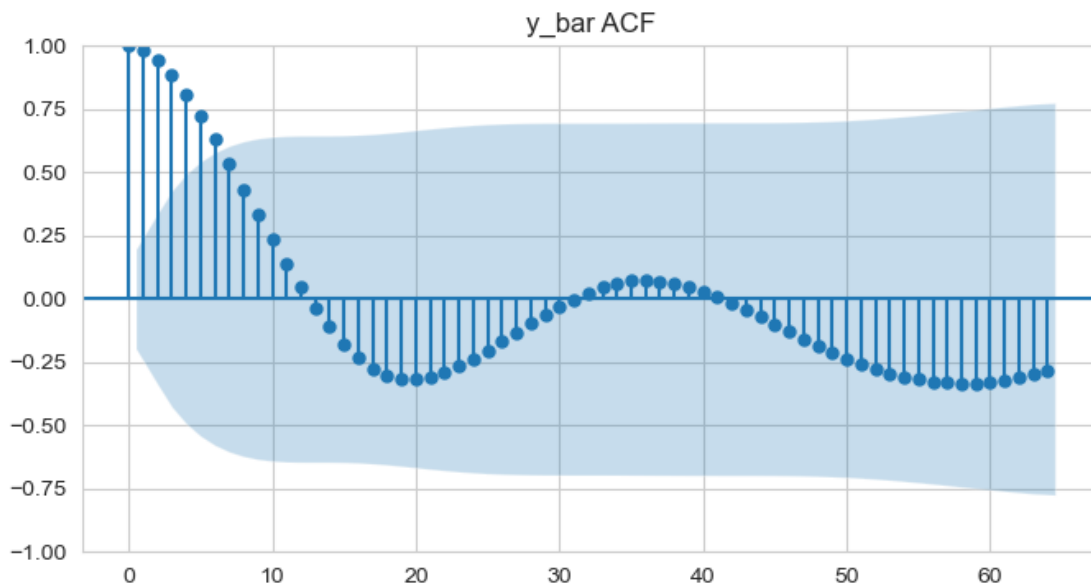
```



```

ACCEL_Z | Estimating ^ | AR(2) | User: wh298
Estimated ^ = [-0.1445 -0.0127]
           = [-0.1445 -0.0127]
Root 1: -0.0722+0.0866j | |z|=0.1128
Root 2: -0.0722-0.0866j | |z|=0.1128
Non-stationary
ACF for Y_BAR | 64 lags (auto)

```



Y_BAR | Estimating $\hat{\cdot}$ | AR(2) | User: wh298

Estimated $\hat{\cdot}$ = [1.9155 -0.9395]

= [1.9155 -0.9395]

Root 1: 0.9578+0.1491j | |z|=0.9693

Root 2: 0.9578-0.1491j | |z|=0.9693

Non-stationary

```
[1199]: {'accel_x': {'phi': array([-0.05110016, -0.23841251]),
  'roots': array([-0.02555008+0.48760609j, -0.02555008-0.48760609j]),
  'moduli': array([0.48827503, 0.48827503]),
  'stationary': False},
  'accel_y': {'phi': array([-0.01231753, -0.15455006]),
  'roots': array([-0.00615876+0.39308031j, -0.00615876-0.39308031j]),
  'moduli': array([0.39312856, 0.39312856]),
  'stationary': False},
  'accel_z': {'phi': array([-0.14446086, -0.01272394]),
  'roots': array([-0.07223043+0.08664125j, -0.07223043-0.08664125j]),
  'moduli': array([0.11280045, 0.11280045]),
  'stationary': False},
  'y_bar': {'phi': array([ 1.91553289, -0.93954101]),
  'roots': array([0.95776644+0.14907866j, 0.95776644-0.14907866j]),
  'moduli': array([0.96929924, 0.96929924]),
  'stationary': False}}
```

```
[1200]: df_diff2 = difference_all_windows(df_windows_seas, order=2)
df_diff2
```

Differenced 112685 windows | order=2

[1200]:

```

                                accel_x  \
0      [-0.009524319999999992, 0.033348899999999996, -0...
1      [-0.05240528999999999, 0.028584809999999996, 0...
2      [0.11910296999999997, -0.07146174, -0.01905633...
3      [0.019056550000000005, -0.0142923500000000065, ...
4      [0.0285848300000000006, -0.038113080000000002, 0...
...
112680 [0.0281159999999999364, -0.009571499999999994, -0...
112681 [-0.68554350000000005, -0.53479579999999995, -0...
112682 [0.115453899999999944, -0.64965179999999996, 0.4...
112683 [-9.99999993922529e-08, -0.0053839000000000441,...
112684 [0.065802699999999988, -0.078963099999999973, 0...

                                accel_y  \
0      [0.047621080000000009, 0.0238104799999999967, -0...
1      [0.038096500000000009, -0.052382649999999975, 0...
2      [-0.057144750000000008, 0.0047620499999999962, 0...
3      [0.11428984999999992, -0.15714847999999992, 0...
4      [0.033334499999999934, 0.0285724500000000083, -...
...
112680 [0.019143999999999894, -0.023928999999999898, 0...
112681 [0.037687999999999928, -0.031705999999999979, -0...
112682 [0.412162600000000116, -1.0935191999999994, -0...
112683 [-0.052043999999999865, 0.032899999999999971, -0...
112684 [-0.00179400000000002954, -0.233899000000000097,...

                                accel_z  \
0      [-0.113916000000000146, 0.0189850000000000696, 0...
1      [0.037973500000000144, 0.0427179999999998923, -0...
2      [0.061702999999999962, 0.061705999999999915, -0...
3      [-0.004744999999999777, 0.009492000000000161, -...
4      [0.033224999999999984, -0.061705999999999915, 0...
...
112680 [-0.024526414999999982, 0.010169489999999999, -...
112681 [0.14476579999999994, 0.120239320000000004, 0.2...
112682 [0.131605199999999976, -0.82013999999999996, -0...
112683 [-0.0095712700000000076, -0.124426819999999983, ...
112684 [-0.23449662799999998, -0.022731791, 0.1770688...

                                y_bar    user  \
0      [4.935826609653304e-07, 4.935826609653304e-07,...  vbagdi
1      [2.877677161450265e-06, 2.734657982728095e-06,...  vbagdi
2      [-4.213305714984017e-06, -4.213305714984017e-0...  vbagdi
3      [-3.730224875653221e-06, -3.730224761966383e-0...  vbagdi
4      [8.907592132345599e-06, 8.907591904971923e-06,...  vbagdi
...
112680 [4.9163196763402084e-05, 4.916319687708892e-05...  xichen
```

```

112681 [-4.368089093986782e-06, -4.3680889802999445e-... xichen
112682 [-4.6080655010882765e-05, -5.822343007366726e-... xichen
112683 [7.88741923543057e-05, 6.68297536776663e-05, 3... xichen
112684 [2.6279559961039922e-05, 2.6279559961039922e-0... xichen

```

	activity	session_id	label	ts_start	\
0	Stationary	vbagdi_Stationary_session1	1	1.761522e+12	
1	Stationary	vbagdi_Stationary_session1	1	1.761522e+12	
2	Stationary	vbagdi_Stationary_session1	1	1.761522e+12	
3	Stationary	vbagdi_Stationary_session1	1	1.761522e+12	
4	Stationary	vbagdi_Stationary_session1	1	1.761522e+12	
...	...	...	...	...	
112680	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12	
112681	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12	
112682	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12	
112683	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12	
112684	Elevator-down	xichen_Elevator-down_session2	7	1.760986e+12	

	ts_end
0	1.761522e+12
1	1.761522e+12
2	1.761522e+12
3	1.761522e+12
4	1.761522e+12
...	...
112680	1.760986e+12
112681	1.760986e+12
112682	1.760986e+12
112683	1.760986e+12
112684	1.760986e+12

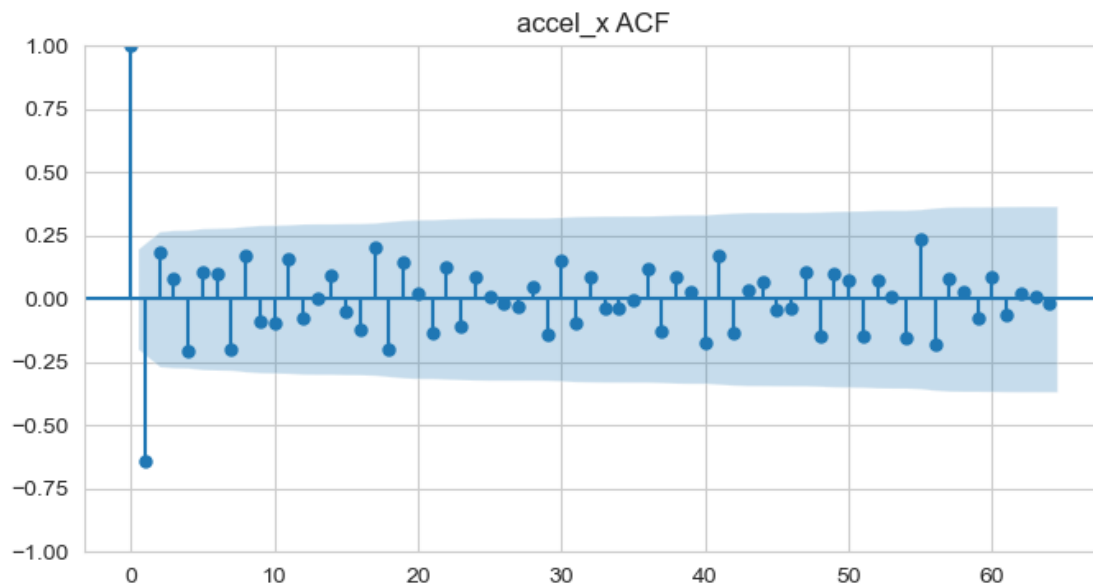
[112685 rows x 10 columns]

```

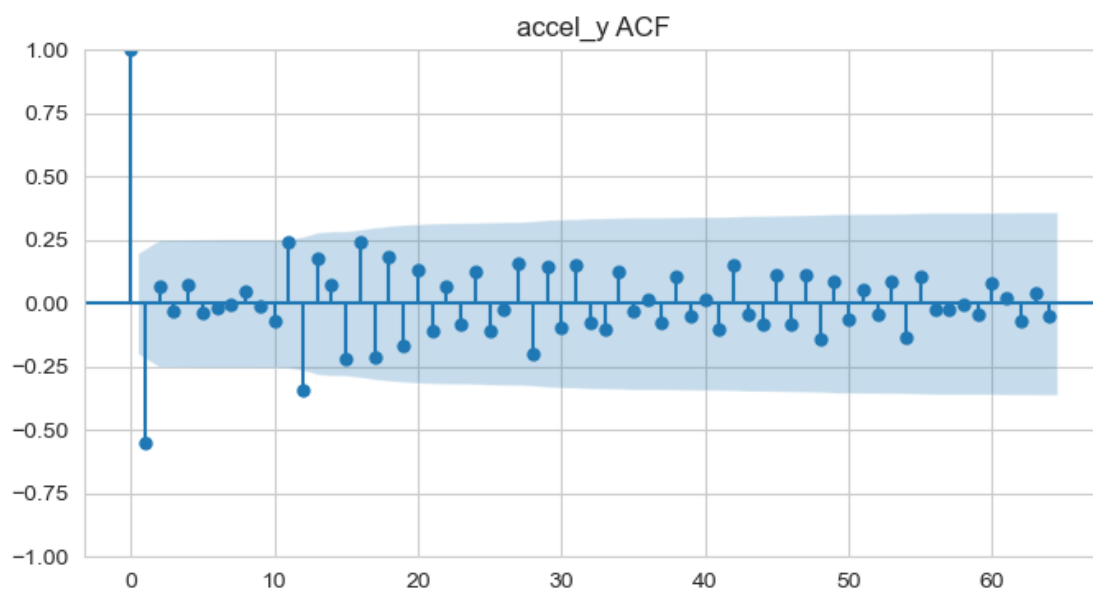
[1201]: results = arma_stationarity_random_window(df_diff2, p=2)
results

```

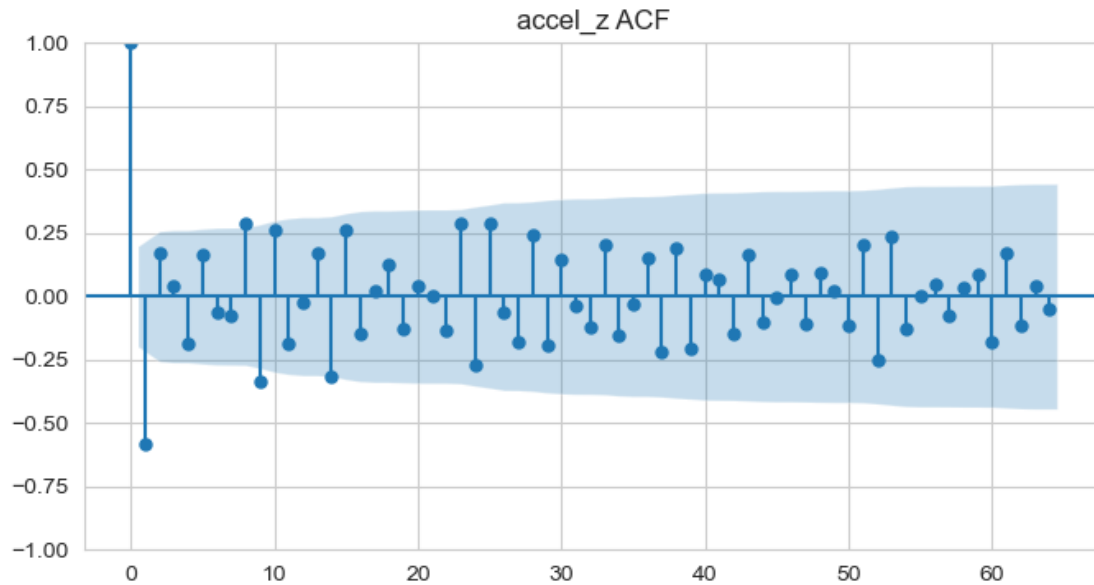
Random window selected from activity: Elevator-down
Randomly selected user: jude
ACF for ACCEL_X | 64 lags (auto)



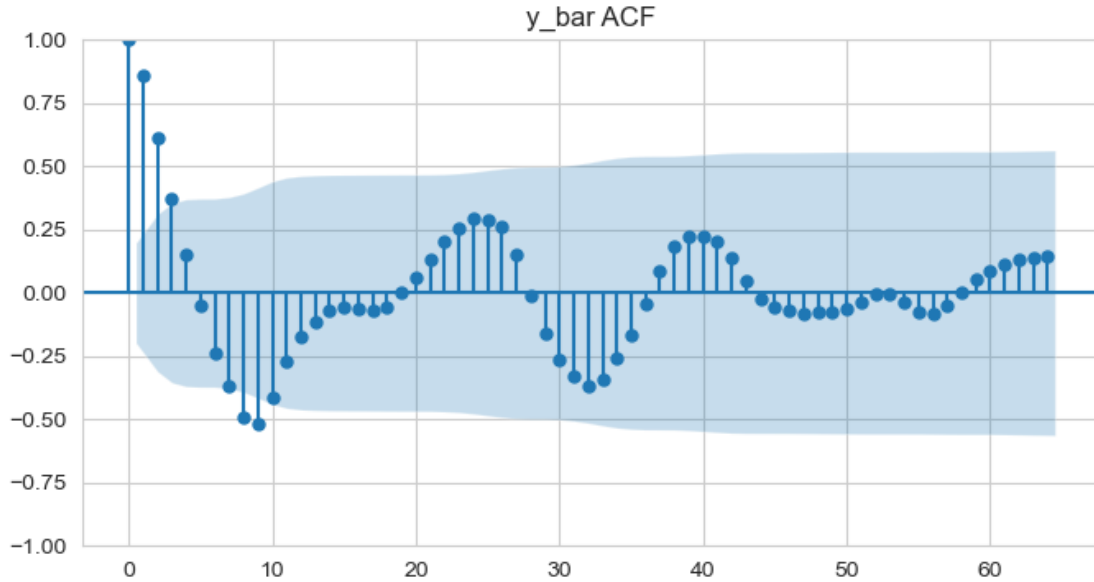
```
ACCEL_X | Estimating ^ | AR(2) | User: jude
Estimated ^ = [-0.9129 -0.4011]
= [-0.9129 -0.4011]
Root 1: -0.4565+0.4390j | |z|=0.6333
Root 2: -0.4565-0.4390j | |z|=0.6333
Non-stationary
ACF for ACCEL_Y | 64 lags (auto)
```



ACCEL_Y | Estimating $\hat{\cdot}$ | AR(2) | User: jude
 Estimated $\hat{\cdot}$ = [-0.753 -0.355]
 = [-0.753 -0.355]
 Root 1: -0.3765+0.4617j | |z|=0.5958
 Root 2: -0.3765-0.4617j | |z|=0.5958
 Non-stationary
 ACF for ACCEL_Z | 64 lags (auto)



ACCEL_Z | Estimating $\hat{\cdot}$ | AR(2) | User: jude
 Estimated $\hat{\cdot}$ = [-0.7373 -0.2601]
 = [-0.7373 -0.2601]
 Root 1: -0.3686+0.3525j | |z|=0.5100
 Root 2: -0.3686-0.3525j | |z|=0.5100
 Non-stationary
 ACF for Y_BAR | 64 lags (auto)



```
Y_BAR | Estimating ^ | AR(2) | User: jude
Estimated ^ = [ 1.2596 -0.4667]
= [ 1.2596 -0.4667]
Root 1: 0.6298+0.2647j | |z|=0.6832
Root 2: 0.6298-0.2647j | |z|=0.6832
Non-stationary
```

```
[1201]: {'accel_x': {'phi': array([-0.91291771, -0.40106626]),
'roots': array([-0.45645885+0.43898927j, -0.45645885-0.43898927j]),
'moduli': array([0.63329792, 0.63329792]),
'stationary': False},
'accel_y': {'phi': array([-0.75303174, -0.35495205]),
'roots': array([-0.37651587+0.4617227j, -0.37651587-0.4617227j]),
'moduli': array([0.59577853, 0.59577853]),
'stationary': False},
'accel_z': {'phi': array([-0.73725117, -0.26014134]),
'roots': array([-0.36862559+0.35250038j, -0.36862559-0.35250038j]),
'moduli': array([0.51004053, 0.51004053]),
'stationary': False},
'y_bar': {'phi': array([ 1.25961045, -0.46672215]),
'roots': array([0.62980523+0.26470271j, 0.62980523-0.26470271j]),
'moduli': array([0.68317066, 0.68317066]),
'stationary': False}}
```

9.1 Test all windows adjusted for seasonality

```
[ ]: # ADF helper
def adf_stationarity(x, alpha=0.05):
    """Return True if stationary by ADF test at given alpha level."""
    x = np.asarray(x, float)
    if len(x) < 8 or np.allclose(x, x[0]):
        return False
    try:
        result = adfuller(x, autolag='AIC')
        pval = result[1]
        return pval < alpha
    except Exception:
        return False

# Worker to compute per-window stationarity
@ray.remote
def check_stationarity_row(row):
    res = {}
    for col in ["accel_x", "accel_y", "accel_z", "y_bar"]:
        res[col] = adf_stationarity(row[col])
    res["activity"] = row["activity"]
    res["user"] = row["user"]
    return res

# Run in parallel
def stationarity_summary_by_activity(df_windows_seas):
    if not ray.is_initialized():
        ray.init(ignore_reinit_error=True)
    rows = [r._asdict() if hasattr(r, "_asdict") else r.to_dict() for _, r in
df_windows_seas.iterrows()]
    results = ray.get([check_stationarity_row.remote(r) for r in rows])
    df_stationarity = pd.DataFrame(results)
    return df_stationarity

[ ]: # Apply to your seasonally differenced windows
df_stat = stationarity_summary_by_activity(df_windows_seas)

# Summarize by activity × sensor
summary = (
    df_stat.melt(id_vars=["activity", "user"], var_name="sensor",
value_name="stationary")
    .groupby(["activity", "sensor"])
    .agg(
        n_windows=("stationary", "count"),
        n_stationary=("stationary", "sum")
    )
)
```

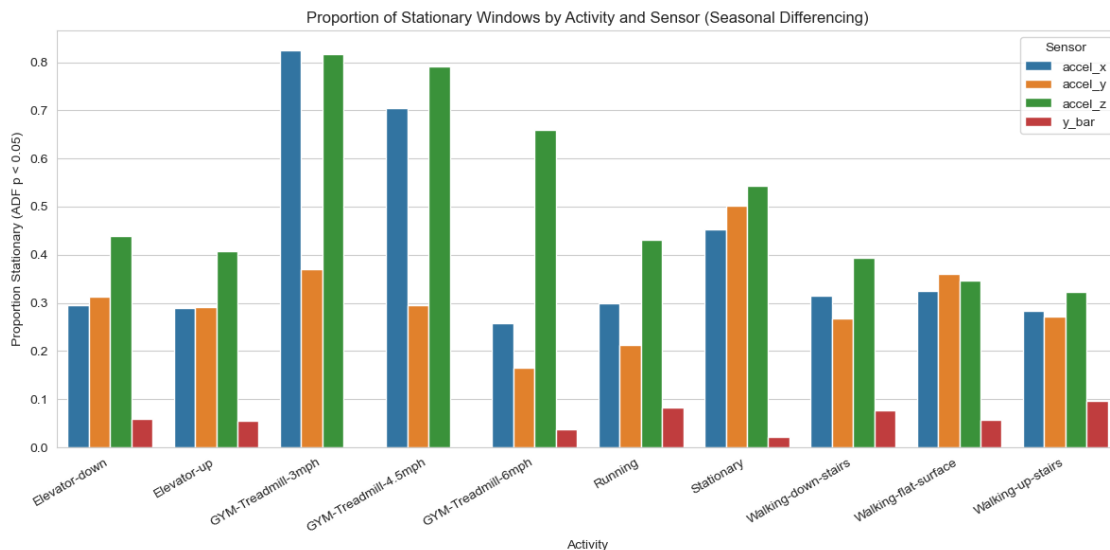
```

        .reset_index()
    )
    summary["prop_stationary"] = summary["n_stationary"] / summary["n_windows"]

    # Visualization
    plt.figure(figsize=(12,6))
    sns.barplot(data=summary, x="activity", y="prop_stationary", hue="sensor")
    plt.title("Proportion of Stationary Windows by Activity and Sensor (Seasonal_
    ↪Differencing)")
    plt.ylabel("Proportion Stationary (ADF p < 0.05)")
    plt.xlabel("Activity")
    plt.xticks(rotation=30, ha="right")
    plt.legend(title="Sensor")
    plt.tight_layout()
    plt.show()

    # pivot table
    summary_pivot = summary.pivot(index="activity", columns="sensor",
    ↪values="prop_stationary")
    summary_pivot

```



```

[ ]: sensor          accel_x  accel_y  accel_z  y_bar
activity
Elevator-down      0.295598  0.313056  0.439756  0.060084
Elevator-up        0.288640  0.290916  0.407382  0.055413
GYM-Treadmill-3mph 0.823970  0.370787  0.816479  0.000000
GYM-Treadmill-4.5mph 0.704120  0.295880  0.790262  0.000000
GYM-Treadmill-6mph 0.257500  0.165000  0.660000  0.037500

```

Running	0.298763	0.213232	0.430413	0.083073
Stationary	0.453590	0.502653	0.543429	0.022586
Walking-down-stairs	0.315824	0.266822	0.393351	0.076130
Walking-flat-surface	0.324623	0.360021	0.345584	0.056498
Walking-up-stairs	0.282743	0.271194	0.323425	0.095801